PRISM

Workflows, specification and build plan

Everything about taking PRISM from a one-off repository scanner to a continuous code-intelligence product: how people use it, what the system does, every feature's rules and data, and the order to build it.

Part	What it covers
1 Workflows	Nine end-to-end workflows: what the user does and what the system does at each step.
2 Specification	The system today (read from the code), shared logic, data model, 26 feature specs, routes, tests, decisions.
3 Build plan	Releases, dependency map, risks, and a task checklist for each of the 16 milestones.

Version	Status	Date	Branch
1.0	Draft for review	2026-10-27	`claude/prism-app-expansion-a0693e`

The same material is on the Figma board in the Alpha team: `figma.com/board/MDgF0WoaULcyqh6hpAhZ9K`

Exists today	New (planned)	Decision point	External service	Warning or failure

Contents

Part 1 - Workflows 5

The whole journey . . . 5

W1 First-time setup . . . 6

W2 Analyze a repository manually . . . 7

W3 Automatic analysis on push and pull request . . . 8

W4 Review and act on a finding . . . 9

W5 History, trend and diff . . . 10

W6 Team lead weekly review . . . 11

W7 Export and share a report . . . 12

W8 Plan limits and billing . . . 13

W9 Failure and recovery . . . 14

Part 2 - Specification 15

1 Product . . . 16

1.1 Where PRISM stands . . . 16

1.2 Goals and non-goals . . . 16

1.3 Product principles (used to settle design questions) . . . 16

1.4 Target product flow . . . 17

2 The system today (read from the code) . . . 18

2.1 Request flow for a repository page . . . 18

2.2 Analysis pipeline . . . 19

2.3 Health score . . . 19

2.4 File risk and hotspots . . . 20

2.5 Sessions, privacy and AI . . . 20

2.6 Gaps found while reading the code . . . 20

3 Shared logic . . . 21

3.1 Stable issue fingerprints . . . 21

3.2 Score as a pure function . . . 21

3.3 Triage overlay . . . 21

3.4 Policy evaluation (PR checks and alerts) . . . 22

3.5 Plans and entitlements . . . 22
3.6 Error codes and what each does . . . 23
3.7 Security rules for every new surface . . . 23
3.8 Target architecture . . . 23

4 Data model . . . 24

4.1 Changes to existing tables . . . 25
4.2 New tables in detail . . . 25
4.3 Migration plan . . . 25

5 Feature specs . . . 26

Phase 0 - Foundations . . . 26

F0.1 Migrations . . . 26
F0.2 Fingerprints and richer issues . . . 26
F0.3 Score module and stored inputs . . . 27
F0.4 Tests and CI . . . 27

Phase 1 - Make findings actionable . . . 27

F1.1 Triage findings . . . 27
F1.2 Diff between analyses . . . 28
F1.3 Score trend . . . 29
F1.4 Code snippet view . . . 30

Phase 2 - Make it continuous . . . 30

F2.1 GitHub App and installation . . . 30
F2.2 Webhooks and job queue . . . 31
F2.3 PR check and comment . . . 33
F2.4 Repo settings and policy . . . 33
F2.5 README badge . . . 34
F2.6 Alerts . . . 34
F2.7 Create GitHub issue from a finding . . . 35

Phase 3 - Use the data . . . 35

F3.1 Exports and SARIF . . . 35
F3.2 Churn x complexity hotspots . . . 36
F3.3 Compare repositories and branches . . . 36
F3.4 Organisation dashboard . . . 37

Phase 4 - Product . . . 37

F4.1 More languages and a real Python parser . . . 37
F4.2 Ask your repo . . . 37
F4.3 AI fix pull requests . . . 38

F4.4 Plans and billing . . . 39
F4.5 Workspaces and teams . . . 39
F4.6 Share links . . . 40
F4.7 Settings, onboarding and error states . . . 40

6 Routes and screens . . . 41
Sidebar after this spec . . . 41

7 Test plan and CI . . . 42

8 Build order and milestones . . . 43
8.1 Release checklist for every milestone . . . 44

9 Decisions and open questions . . . 44
Decisions this spec makes . . . 44
Questions for you . . . 44

Part 3 - Build plan 45

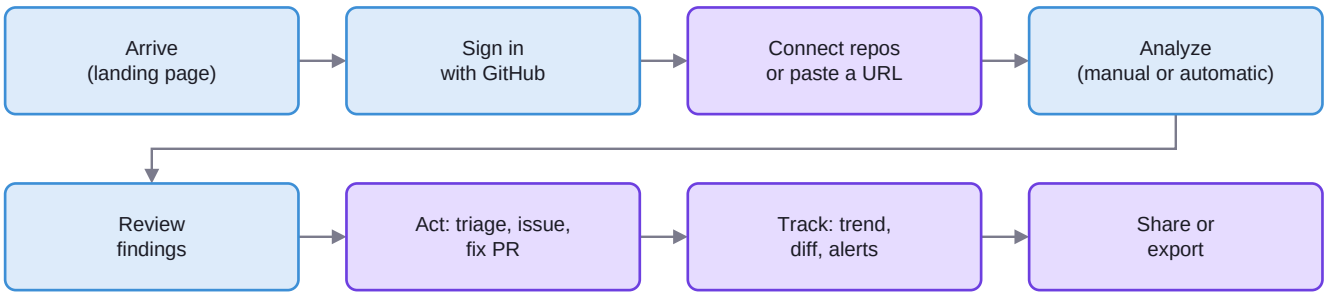
Plan and releases . . . 45
Releases . . . 45
Dependency map . . . 45
Risks to watch . . . 46
Definition of done for every milestone . . . 46

Milestone checklists . . . 46
M0 Foundations . . . 46
M1 Triage . . . 47
M2 Diff and trend . . . 47
M3 Code snippets . . . 47
M4 GitHub App, webhooks, jobs . . . 48
M5 PR check and comment . . . 48
M6 Alerts and badge . . . 48
M7 Issues from findings . . . 49
M8 Exports and SARIF . . . 49
M9 Churn hotspots . . . 49
M10 Compare and org dashboard . . . 49
M11 Plans and billing . . . 50
M12 Workspaces and share links . . . 50
M13 More languages . . . 50
M14 Ask your repo . . . 50
M15 AI fix PRs . . . 51

Part 1 - Workflows

How people use PRISM and what the system does at each step. Diagrams show the main path; branches at decision points are described in the step table under each one.

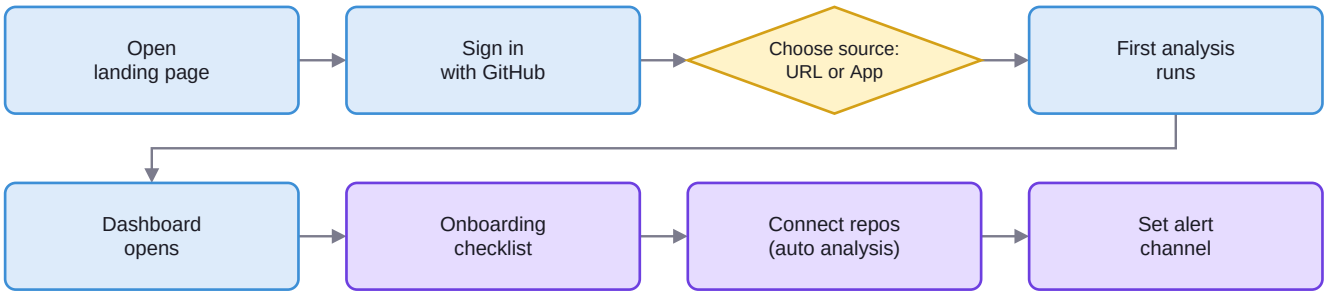
The whole journey



#	Workflow	Who	When	Outcome
W1	First-time setup	New user	First visit	A first analysis, a populated dashboard and a connected repository.
W2	Analyze a repository manually	Any user	Any time	A saved analysis and a dashboard for the repository.
W3	Automatic analysis on push and pull request	Developer (GitHub)	Every push to the default branch and every pull request update	A check and a single comment on the pull request, a saved analysis, and alerts if needed.
W4	Review and act on a finding	Developer	After an analysis, from any issue list	Each finding is fixed, tracked or consciously set aside, and the score reflects it.
W5	History, trend and diff	Developer or lead	To see what changed between two runs	A clear list of new, fixed and unchanged issues and the score movement.
W6	Team lead weekly review	Team lead or maintainer	Weekly	Attention goes to the repositories that got worse; owners are assigned.
W7	Export and share a report	Any signed-in user	When results must leave PRISM	A file or a read-only link, with secrets masked.
W8	Plan limits and billing	User or team owner	When a limit is reached or a plan changes	The right features are on, and nothing is lost when a plan ends.
W9	Failure and recovery	System, then user	When an analysis cannot complete	The user always knows what happened and what to do; transient problems fix themselves.

W1 First-time setup

Who	When	Outcome
New user	First visit	A first analysis, a populated dashboard and a connected repository.



Step by step

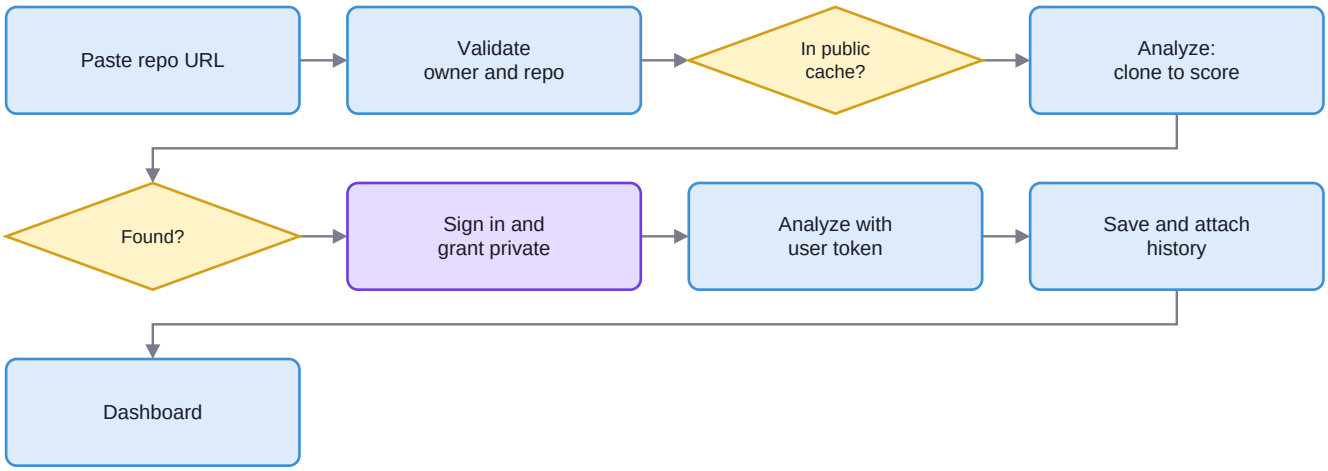
#	Who	What happens
1	User	Opens the landing page and presses Open app.
2	PRISM	Starts GitHub sign-in asking only for the public profile. GitHub's account picker is shown every time.
3	User	Chooses a source: paste a public repository URL, or install the GitHub App to connect repositories for automatic analysis.
4	PRISM	Runs the analysis (clone, parse, secrets, graph, dependencies, quality, score) with a progress screen.
5	PRISM	Opens the dashboard: health score, 3D map, top risks and the AI summary.
6	PRISM	Shows a three-step checklist: connect repositories, triage the first findings, add an alert channel. Each step can be skipped.
7	User	Connects repositories and chooses which ones get automatic analysis.
8	User	Adds a Slack or email channel and sends a test message.

What can go wrong, and what the user sees

- Sign-in cancelled or expired link: back to Setup with a plain message, nothing is stored.
- Repository not found: offer sign-in (private repo) or a spelling check.
- GitHub rate limit: show when it resets; the user can try again then.

W2 Analyze a repository manually

Who	When	Outcome
Any user	Any time	A saved analysis and a dashboard for the repository.



Step by step

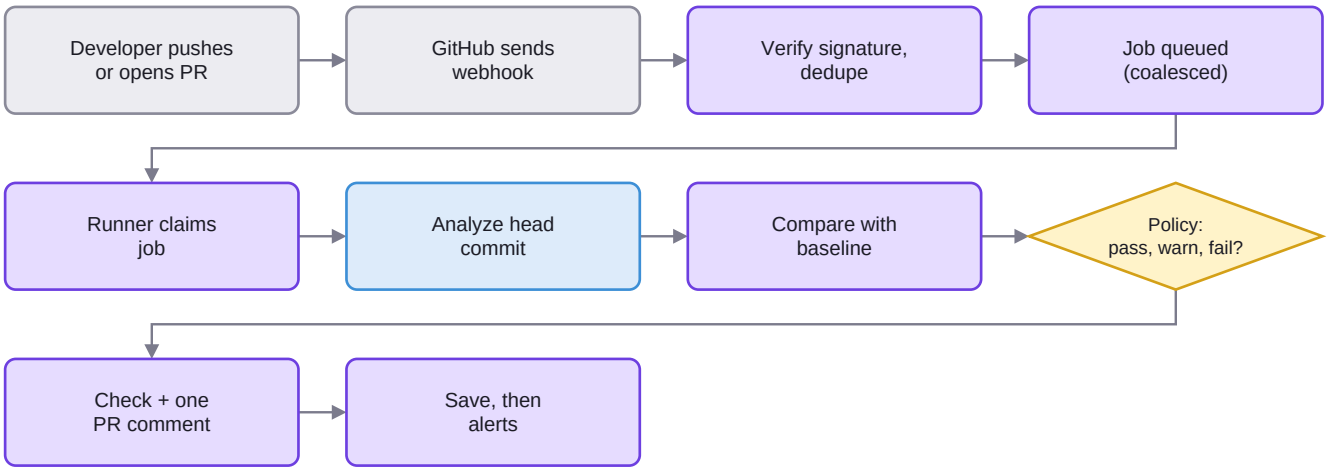
#	Who	What happens
1	User	Pastes owner/repo or a GitHub URL on the New analysis page.
2	PRISM	Checks the name format. A bad name shows a friendly error immediately.
3	PRISM	Looks in the shared public cache (kept for hours). A hit opens the dashboard at once.
4	PRISM	On a miss, downloads the default branch and runs all analysis stages.
5	PRISM	If GitHub says not found, the repo may be private: signed-out users are asked to sign in; signed-in users without the repo scope are asked to allow private access.
6	User	Allows private access (a separate GitHub consent screen). PRISM never stores the token in the browser.
7	PRISM	Analyzes with the user's own token. Private results never enter the shared cache; a result under 30 minutes old is reused.
8	PRISM	Signed-in users get the analysis saved and the last scores attached for the trend.
9	PRISM	Opens the dashboard.

What can go wrong, and what the user sees

- Repo over the size limits: explain the limits and what was skipped.
- No supported source files: say which languages are supported today.
- Dependency check unavailable: analysis continues, with a note.

W3 Automatic analysis on push and pull request

Who	When	Outcome
Developer (GitHub)	Every push to the default branch and every pull request update	A check and a single comment on the pull request, a saved analysis, and alerts if needed.



Step by step

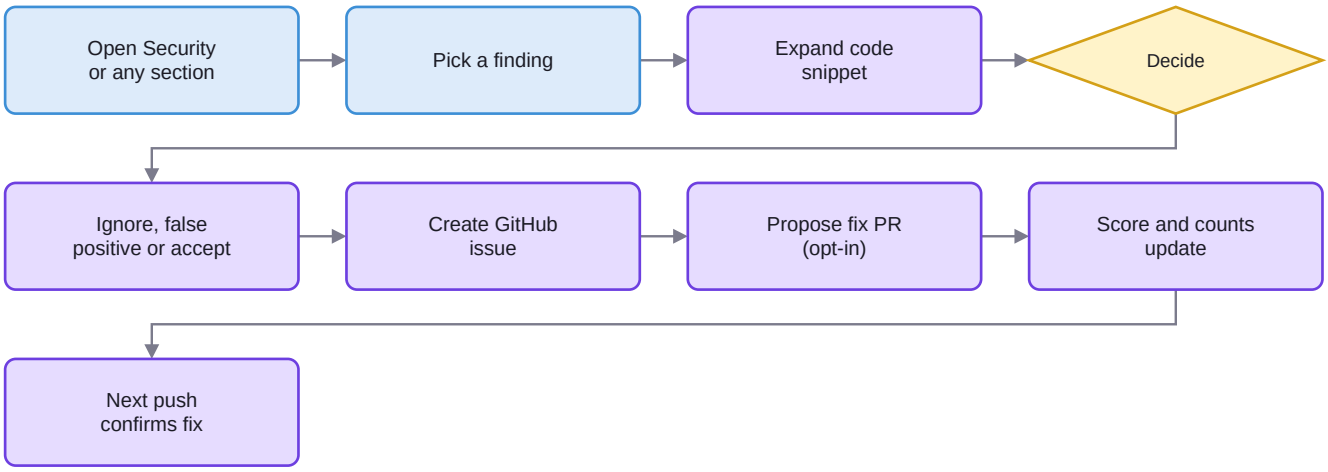
#	Who	What happens
1	Developer	Pushes a commit or updates a pull request.
2	GitHub	Sends a signed webhook to PRISM.
3	PRISM	Verifies the signature, ignores a delivery it has already seen, and answers in under a second.
4	PRISM	Queues a job. Older queued jobs for the same branch are canceled, so ten quick pushes cost one analysis.
5	PRISM	A runner (started every minute) claims the job and marks the PR check as in progress.
6	PRISM	Analyzes the new commit. For a PR it also needs the base commit's analysis; if missing, that is analyzed first.
7	PRISM	Compares the two by issue fingerprint, so only genuinely new problems count. Triaged issues are excluded.
8	PRISM	Applies the repository policy: a new secret or new critical fails; a new high warns; otherwise pass.
9	PRISM	Completes the check with a title such as 'Health 82 to 76 (-6): 2 new critical', adds annotations on the changed lines, and updates one PR comment.
10	PRISM	Saves the analysis, then sends Slack or email alerts if a rule matches, at most one per repository per hour.

What can go wrong, and what the user sees

- Fork pull request with read-only access: neutral check saying the fork cannot be read.
- Baseline missing after two tries: run without it; the comment says so and the check is neutral.
- GitHub or analysis errors: retried after 1, 5 and 15 minutes; then marked failed and shown in History.

W4 Review and act on a finding

Who	When	Outcome
Developer	After an analysis, from any issue list	Each finding is fixed, tracked or consciously set aside, and the score reflects it.



Step by step

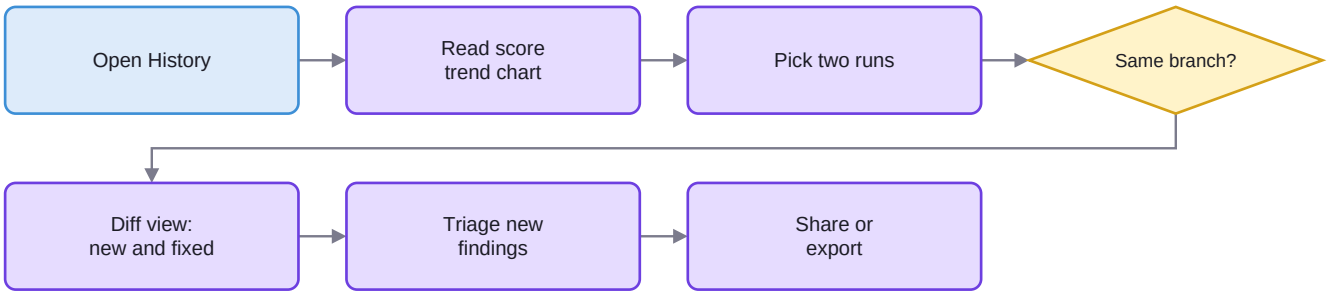
#	Who	What happens
1	User	Opens a section and picks a finding (sorted by risk).
2	PRISM	Expanding it loads the offending lines with context, from the analysed commit. Secret values are always masked.
3	User	Reads the AI explanation and the 'might be a false positive' rating.
4	User	Decides: set it aside (ignore, false positive, accept risk with an expiry), track it (create a GitHub issue), or ask for a fix.
5	PRISM	Set aside: the finding moves to the Triaged tab and the score recalculates immediately. Accepted risk reopens when its expiry passes.
6	PRISM	Track: previews the issue (title, permalink, impact, AI fix) and creates it once; the finding shows 'Tracked: #123'.
7	PRISM	Fix PR (only if the repo allows sending a snippet to the AI): proposes a patch, checks it applies and no longer triggers the rule, then opens a pull request. Never merges.
8	PRISM	On the next analysis, a finding that no longer appears is marked fixed; a linked issue gets a comment, and is closed only if that is switched on.

What can go wrong, and what the user sees

- Signed out: triage and issue buttons are hidden with a hint to sign in.
- Secret findings: no fix PR; the guidance is to rotate the key.
- Triage is per person: it never changes what another user sees.

W5 History, trend and diff

Who	When	Outcome
Developer or lead	To see what changed between two runs	A clear list of new, fixed and unchanged issues and the score movement.



Step by step

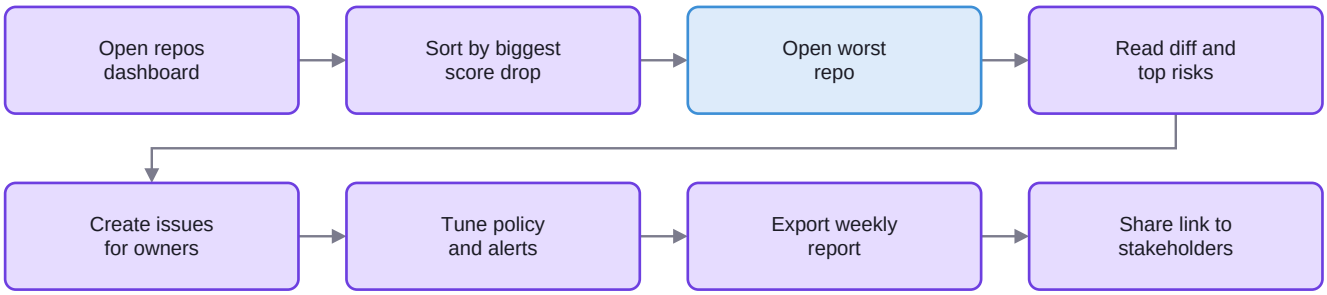
#	Who	What happens
1	User	Opens History and sees a chart of the score over time with a row per run.
2	PRISM	The chart shows the triage-adjusted score by default; a toggle shows the raw score.
3	User	Selects two runs (default: latest against the previous run on the same branch) or presses Compare on a row.
4	PRISM	Checks they are comparable (same branch, scoring version). If not, shows a banner and still compares.
5	PRISM	Shows score change, then New, Fixed, Changed and Unchanged issues and metric changes.
6	User	Triages new findings inline, or opens a finding's snippet.
7	User	Exports the diff or shares the report.

What can go wrong, and what the user sees

- First analysis of a repo: nothing to compare yet.
- Older runs without fingerprints: compared approximately, and labelled so.
- Results capped at 300 issues: the diff says it may be approximate.

W6 Team lead weekly review

Who	When	Outcome
Team lead or maintainer	Weekly	Attention goes to the repositories that got worse; owners are assigned.



Step by step

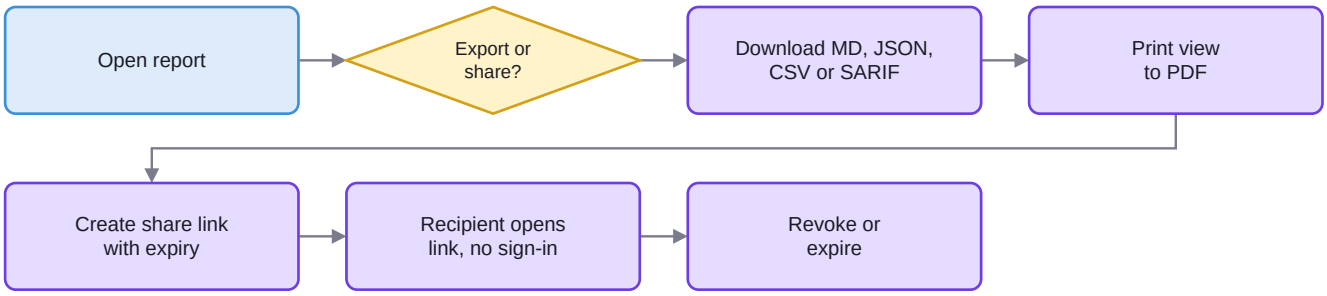
#	Who	What happens
1	User	Opens the repositories dashboard: score, trend, critical count and last-analysed time for every repo.
2	PRISM	Highlights repos below the threshold, not analysed in 30 days, or with a failing run.
3	User	Sorts by biggest score drop this week and opens the worst repository.
4	User	Reads the diff against last week and the top risks and hotspots (complex and frequently changed files).
5	User	Creates GitHub issues for the findings that need an owner.
6	User	Adjusts the repository policy (fail on critical, minimum score, maximum drop) and alert rules, using the preview of how the last ten PRs would have fared.
7	User	Exports a report (print to PDF, Markdown or CSV).
8	User	Creates a share link with an expiry for people who do not use PRISM.

What can go wrong, and what the user sees

- Many repos: the list is paginated and totals are computed in the database.
- Different scoring versions between repos: a banner explains the comparison.

W7 Export and share a report

Who	When	Outcome
Any signed-in user	When results must leave PRISM	A file or a read-only link, with secrets masked.



Step by step

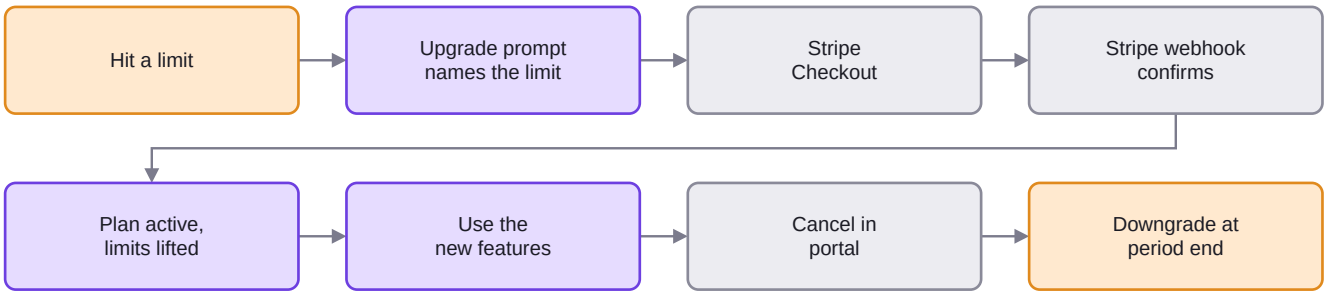
#	Who	What happens
1	User	Opens a repository report and presses Export or Share.
2	User	Export: picks a format. SARIF can also be sent to the GitHub Security tab.
3	PRISM	Builds the file from the saved analysis. Secret values are masked in every format; triaged issues keep their state.
4	User	Print view: opens a print-ready report and saves it as PDF from the browser.
5	User	Share: chooses 24 hours, 7 days or 30 days. For private repos a warning appears and file names are hidden unless ticked.
6	Recipient	Opens the link without signing in and sees a read-only snapshot of that single analysis.
7	User	Revokes the link from the list, or lets it expire. It stops working within about a minute.

What can go wrong, and what the user sees

- Large SARIF (over 10 MB or 25,000 results): trimmed to the highest risks with a note.
- Free plan: share links limited or unavailable, with an upgrade prompt.

W8 Plan limits and billing

Who	When	Outcome
User or team owner	When a limit is reached or a plan changes	The right features are on, and nothing is lost when a plan ends.



Step by step

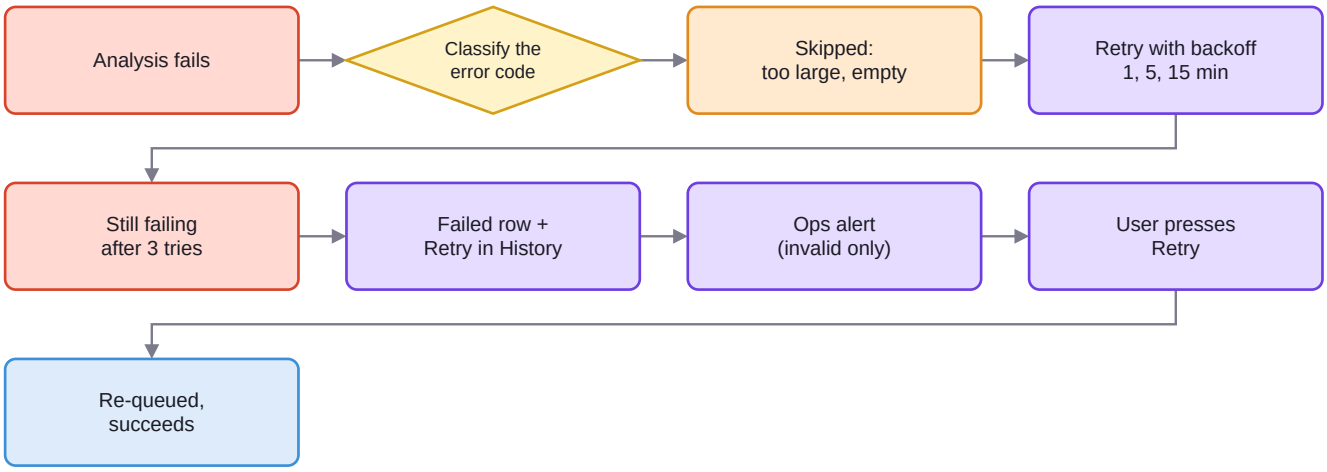
#	Who	What happens
1	User	Reaches a limit (for example a fourth private repository).
2	PRISM	Shows which limit was reached, the current usage and what each plan includes.
3	User	Chooses a plan and pays on Stripe's hosted page.
4	Stripe	Sends a signed event; PRISM verifies it and records the subscription once (replays change nothing).
5	PRISM	Applies the plan immediately; the user returns to the page they started from.
6	User	Uses the features: automatic analysis, PR checks, alerts, more AI messages.
7	User	Cancels or changes the card in Stripe's customer portal.
8	PRISM	At period end returns to Free: automatic analysis pauses and extra private repos become read-only. Nothing is deleted.

What can go wrong, and what the user sees

- Payment fails: the plan stays for 7 days with a banner, then ends.
- Webhook arrives before the user returns: both paths use the same record.

W9 Failure and recovery

Who	When	Outcome
System, then user	When an analysis cannot complete	The user always knows what happened and what to do; transient problems fix themselves.



Step by step

#	Who	What happens
1	PRISM	An analysis fails: GitHub unreachable, rate limited, repo too large or empty, or an unexpected error.
2	PRISM	Classifies it. Too large or empty is final: the run is marked skipped with the reason.
3	PRISM	Network and rate-limit errors are retried automatically after 1, 5 and 15 minutes. A run stuck for over 10 minutes is requeued.
4	PRISM	After the third failure the run is marked failed and appears in History as a red row with the error text.
5	PRISM	Unexpected errors also raise an alert for the operator; the others do not.
6	User	Presses Retry on the failed row once the cause is fixed (for example the rate limit has reset).
7	PRISM	Re-queues it at normal priority; on success the row turns green and the PR check, if any, is updated.

What can go wrong, and what the user sees

- Interactive page views show the same message immediately instead of waiting for retries.
- The shared cache keeps failures for only a few minutes so a fixed problem is not hidden.

Part 2 - Specification

The system today, the shared logic every feature relies on, the data model, every feature spec, routes, tests and decisions.

1 Product

1.1 Where PRISM stands

Sign in with GitHub, paste a repository, and PRISM downloads the default branch, parses JavaScript, TypeScript and Python, scans for committed secrets, audits dependencies against OSV.dev, builds an import graph and produces a health score, a 3D map and an AI summary. Results are saved per commit for signed-in users. Private repositories work when the user grants the `repo` scope.

The gap. Each analysis is manual, one-off and read-only. Nothing brings people back, nothing tells them what changed, and a finding cannot be acted on, silenced or tracked.

1.2 Goals and non-goals

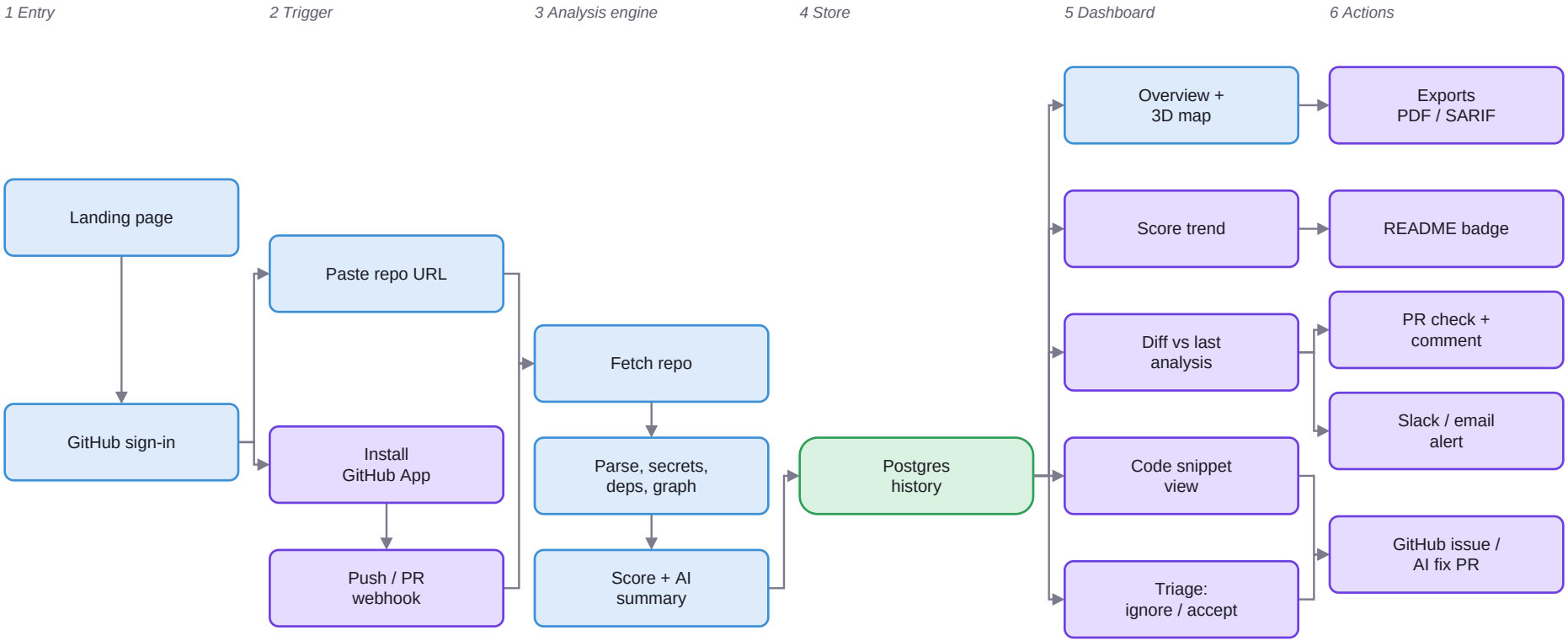
Goals	Non-goals (for now)
- Make repeat visits useful: what changed, what to do next. - Run without being asked: every push and pull request. - Let people act: triage, issue, fix PR, export. - Stay honest: static findings can be wrong, so say how sure we are. - Keep source code private: AI sees facts, not code, unless the user opts in per action.	- Replacing a full SAST suite or a code host. - Auto-merging anything. - Running user code or tests. - Supporting every language: add a few well, not many badly. - Real-time collaboration.

1.3 Product principles (used to settle design questions)

- **Evidence over opinion.** Every number or claim links to an issue id, a file and a line.
- **New problems matter more than old ones.** Checks and alerts fire on what a change introduced, not on the backlog.
- **Degrade, never crash.** Each integration checks its keys; a missing service reduces features, not the whole app (already true in `src/server/config.ts`).
- **Per-user data stays per-user.** Triage and private results never enter the shared public cache.
- **No surprises with permissions.** Ask for the smallest scope first; ask for more only when a feature needs it, and say why.

1.4 Target product flow

Blue is built today, violet is new. Read left to right: how people arrive, what triggers an analysis, what the engine does, where results live, what people see and what they can do.

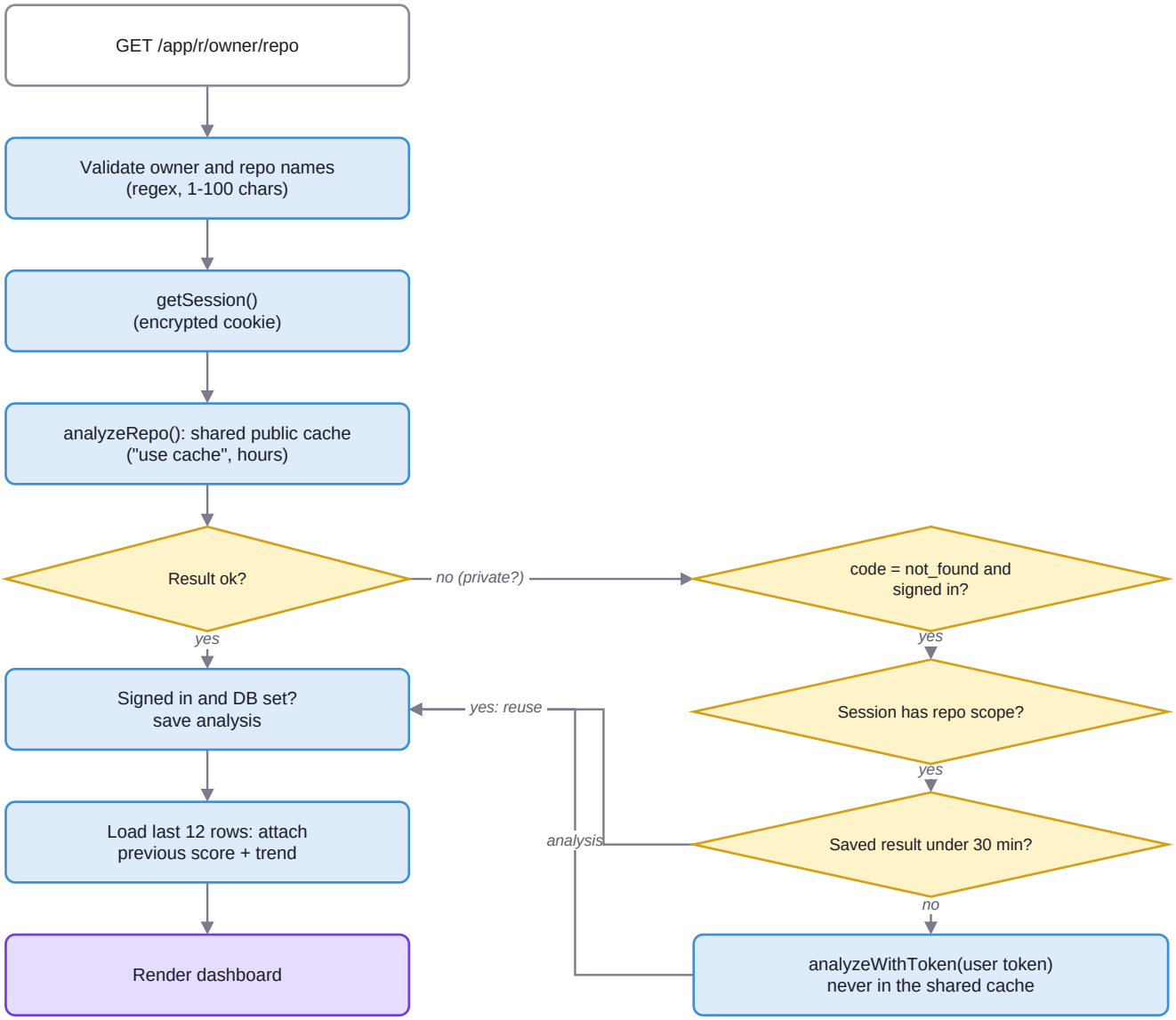


2 The system today (read from the code)

This section records what the code actually does, so the new logic builds on facts. File names are in `src/`.

2.1 Request flow for a repository page

Implemented in `server/loadRepo.ts`. Public repositories are served from a shared cache; private ones use the viewer's own token and never touch that cache.



What the branches mean

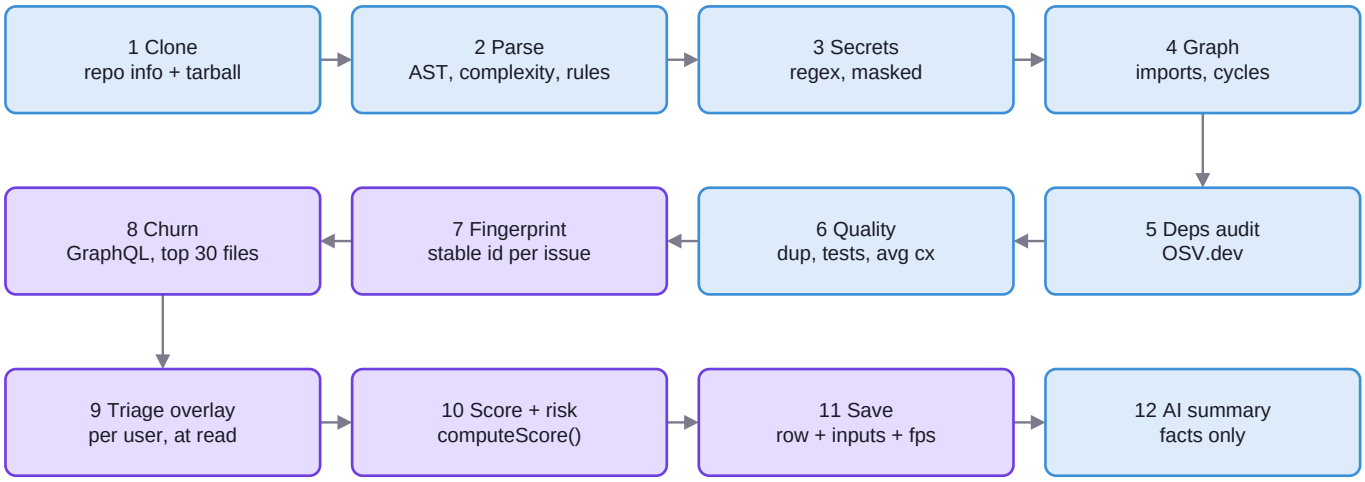
- **Validate:** owner and repo must match `[\w.-]{1,100}`, otherwise a friendly error.
- **Public cache:** `analyzeRepo(owner, repo)` is a `"use cache"` function keyed by lower-cased owner/repo, cached for hours on success and minutes on failure. It is *not* keyed by commit.
- **not_found + no session:** show the error with a sign-in button (only if GitHub auth is configured).
- **not_found + session without `repo` scope:** offer to grant private access (separate OAuth round trip with `?private=1`).
- **Saved result under 30 minutes:** reuse it, regardless of whether the repo changed. Fine for manual use, wrong for automation (see 2.6).

- **Save and history:** `saveAnalysis` inserts one row per (user, repo, commit) and ignores duplicates; `withHistory` attaches the last 12 scores and the previous score.

2.2 Analysis pipeline

Stage	File	What it does	Limits
Clone	fetchRepo.ts	Repo info + tarball of default branch, unpacked in memory. 2 GitHub calls.	150 MB repo, 6,000 files, 40 MB source, 400 KB per file
Parse	parse.ts	JS/TS via TypeScript compiler; Python line by line. Functions, cyclomatic complexity, classes, imports, risky patterns.	JS, TS, Python only
Secrets	secrets.ts	AWS, GitHub, Stripe, Slack, Google keys, private keys, hardcoded passwords, committed .env. Values always masked.	Regex; can false-positive
Graph	graph.ts	Resolves imports into a graph; fan-in per file; circular dependencies.	Top 700 files drawn
Deps	deps.ts	package.json, lockfile, requirements.txt versus OSV.dev.	Network; skipped with a note on failure
Quality	analysis/index.ts	Average complexity, long functions, duplication (6-line windows), test ratio.	Tests, examples, docs excluded
Score	analysis/index.ts	Penalty formula (2.3) and per-file risk (2.4).	Issues capped at 300
AI	ai.ts	Claude (or Gemini) explains facts; cites issue ids; cached per repo + commit for days.	Never sees source code

Pipeline after this spec



Stages 7 to 11 are new: fingerprinting, churn, the triage overlay, a pure scoring function and a richer saved row.

2.3 Health score

Starts at 100 and subtracts a penalty, rounded and clamped to 0 to 100. Each term has its own cap so one noisy category cannot zero the score.

Term	Formula	Cap
Critical issues	`crit * 8`	40
High issues	`high * 25 / (high + 10)`	25
Medium issues	`med * 15 / (med + 20)`	15

Term	Formula	Cap
Average complexity	`(avgCx - 4) * 3`	12
Duplication	`dupPct - 3`	8
Circular dependencies	`cycles * 2`	10

Grades: A 93+, A- 88, B+ 83, B 77, B- 70, C+ 65, C 58, D 50, F below. Severity base risk is 88 / 64 / 40 for critical / high / medium, plus up to 10 for how many files import the affected file, plus 6 for secrets, capped at 99.

2.4 File risk and hotspots

Per file today: `complexity share * 45 + issue weight (max 40) + fan-in share * 15`, capped at 99, where issue weight adds 25 / 12 / 5 per critical / high / medium finding. The top 5 production files become hotspots. Churn is not used yet (the type has a field for it).

2.5 Sessions, privacy and AI

- **Session:** AES-256-GCM encrypted JWE in an HTTP-only cookie, 180 days; holds the GitHub token, never exposed to browser JavaScript.
- **OAuth:** state cookie compared in constant time; return path restricted to same-site paths; `prompt=select_account` shown every time.
- **Delete account:** POST only, deletes the user (cascade removes analyses), revokes the GitHub grant, clears cookies.
- **AI:** facts are the top 10 issues, hotspots, metrics and notes; the model must cite ids; a fallback model handles safety refusals; failures are not cached.
- **Database:** Supabase Postgres via the pooler, RLS on with no policies (the public API cannot read it); schema created on first use.

2.6 Gaps found while reading the code

These are not complaints; they are the places where new features would break if left as they are. Each has a fix in this spec.

#	What the code does	Why it matters	Fix
G1	Issue id = hash of path, line, rule, title (shortId).	Any edit above an issue changes its line, so its id changes. Diff and triage would show phantom new and fixed issues.	Stable fingerprints without line numbers (3.1).
G2	Commit stored as 7 characters.	Fetching a file at a commit and linking to GitHub need the full sha.	Store both; use full sha for APIs.
G3	Issues sorted by risk and cut at 300; complexity capped at 15, large files 5, cycles 10.	When issue 301 moves up after a fix it looks 'new'.	Record overflow count; mark diffs approximate when above zero (F1.2).
G4	Private results reused for 30 minutes whatever the commit.	A webhook run on a fresh commit would get a stale result.	Key reuse by commit sha; the 30 minute rule stays for manual views only.
G5	Public cache keyed by repo, not commit.	Fine for browsing; PR analysis must be per commit and per user token.	Automation bypasses the shared cache.
G6	Schema is created by running SQL on first use.	Cannot express ALTERs safely or roll forward.	Numbered migrations with a ledger (F0.1).
G7	History rows store only counts as the message; 'previous' is the latest row with a different commit.	Ignores branches; cannot say what changed.	Store branch, fingerprints and inputs; compare same branch only.
G8	Hotspot churn and fan-in fields exist; churn never filled.	The 'riskiest files' list ignores how often files change.	Churn via GraphQL for top files (F3.2).

#	What the code does	Why it matters	Fix
G9	No tests anywhere.	Rules, scoring and parsers will regress as logic grows.	Vitest and CI before new features (F0.4).
G10	Analyses belong to one user (user_id not null).	Webhook runs and teams need a workspace owner.	Workspaces (F4.5); until then attribute to the installer.

3 Shared logic

Rules every feature depends on. Specified once, here, so features can refer to them.

3.1 Stable issue fingerprints

A fingerprint identifies *the same problem* across commits, even when lines move. Triage, diff, alerts, SARIF and PR annotations all key on it.

Rule family	Subject (what makes it 'the same')
Pattern rules (eval, SQL by string, innerHTML, N+1, blocking sync, pickle, yaml)	Rule id + file path + enclosing function name + whitespace-normalised source line text
Complexity	Rule id + file path + function name (not the number: it changes)
Large file	Rule id + file path
Cycle	Rule id + sorted list of module paths in the cycle
Secret	Rule id + file path + secret type + hash of the masked value shape (never the value)
Vulnerable dependency	Rule id + ecosystem + package + advisory id (file path only for monorepo manifests)

`fingerprint = sha1(rule | path | subject | occurrence).slice(0, 12)`, where occurrence counts earlier identical (rule, path, subject) triples in file order, so two identical lines in one function stay distinct. The old `PR-xxxxx` id stays as the display id; the fingerprint is a new field, `Issue.fingerprint`, plus `Issue.rule` and `Issue.subject` which parsers already know.

> **Decision.** Fingerprints exclude line numbers and counts. The trade-off: renaming a function makes its issues look fixed and new. That is acceptable and visible in the diff as 'moved or renamed' only if a later version adds similarity matching; v1 does not.

3.2 Score as a pure function

Move the penalty maths out of `runAnalysis` into `src/lib/score.ts`: `computeScore({ crit, high, med, avgCx, dupPct, cycles }) -> { score, grade }`. The saved row stores these inputs and a compact list of fingerprints (`inputs jsonb`: inputs plus `[fp, severity, category, rule]` per issue). Two things follow:

- **Triage-adjusted score:** recompute from inputs with triaged fingerprints removed from the counts (and cycles), without loading the large `result` column.
- **Honest trends:** the trend chart plots adjusted scores for all rows. Triage applies retroactively: it answers 'what would the score have been if we had always ignored this?'. A toggle shows raw scores.
- **Versioning:** `scoreVersion` is stored; if the formula changes, old rows are flagged 'older scoring' instead of silently compared.

3.3 Triage overlay

Triage is applied when a result is read, never inside the analysis, so the shared public cache stays user-neutral.

State	In issue lists	In score and counts	In PR checks and alerts
open	Shown	Counted	Counted

State	In issue lists	In score and counts	In PR checks and alerts
ignored	Under 'Triaged'	Not counted	Not counted
false_positive	Under 'Triaged'; feeds rule feedback	Not counted	Not counted
accepted	Under 'Triaged', with expiry date	Not counted until expiry	Not counted until expiry
fixed (automatic)	Hidden; shown in diff as fixed	n/a	n/a

3.4 Policy evaluation (PR checks and alerts)

Inputs: head analysis H, baseline analysis B, per-repo settings, triage states. *New* issues are fingerprints in H but not in B, minus triaged ones. Rules run in order and the first match wins:

#	Condition	Check conclusion	Setting
1	A new secret exists	failure	`block_secrets` (default on)
2	At least one new critical issue	failure	`fail_on_critical` (default on)
3	Adjusted score of H is below the minimum	failure	`min_score` (default off)
4	Score dropped by at least N versus B	failure	`max_drop` (default off)
5	Any new high issue	neutral (shown as warning)	always
6	Baseline missing	neutral, comment says 'no baseline'	always
7	Otherwise	success	always

Pre-existing issues never fail a check. This is what makes the tool adoptable on old codebases.

3.5 Plans and entitlements

One function, `can(workspace, feature, usage)` in `src/server/plans.ts`, answers every limit question; no feature checks plan names directly. Numbers below are a **proposal** to be decided (see open decisions).

Feature	Free	Pro	Team
Public repo analysis	Unlimited	Unlimited	Unlimited
Private repos	3	Unlimited	Unlimited
Automatic analysis (webhooks)	No	10 repos	Unlimited
PR check and comment	No	Yes	Yes
History retention	30 days	1 year	2 years
Alerts (Slack, email)	No	Yes	Yes
Ask your repo (messages per day)	10	100	300 per seat
AI fix PRs per month	0	20	100
Share links	No	10 active	Unlimited
Seats	1	1	Per seat

Downgrade behaviour: nothing is deleted; automatic analysis pauses, extra private repos become read-only, history older than the window is hidden but kept for 30 days in case the user returns.

3.6 Error codes and what each does

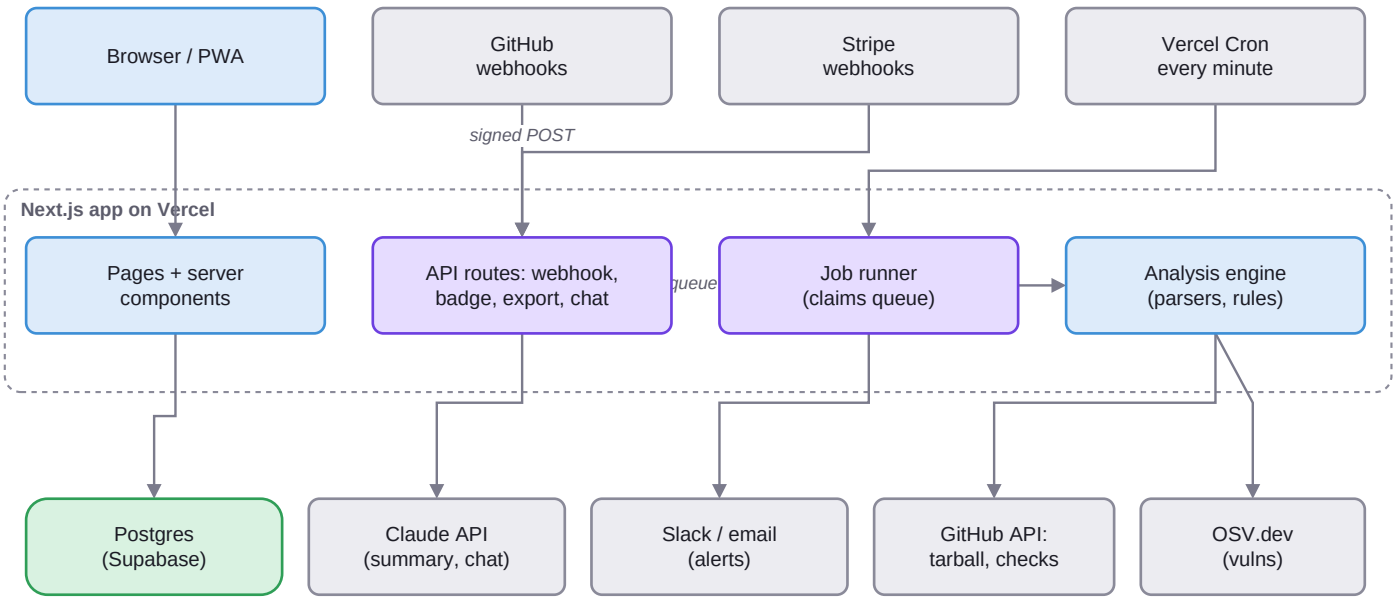
Code	User sees	Interactive page	Background job	Alert?
not_found	Repository not found, or private	Offer sign-in or private access	Mark skipped; check 'neutral'	No
too_large	Repo over limits	Explain limits	Skipped; comment says why	No
rate_limited	GitHub limit reached	Show retry time	Requeue with backoff 1, 5, 15 min	After 3 failures
network	Could not reach GitHub	Retry button	Requeue with backoff	After 3 failures
empty	No supported source files	Explain languages supported	Skipped; check 'neutral'	No
invalid	Something went wrong	Retry; log id	Failed after 3 attempts	Yes (ops)

3.7 Security rules for every new surface

- **Webhooks:** verify `X-Hub-Signature-256` (HMAC-SHA256 of the raw body) in constant time before parsing; reject on mismatch with 401; dedupe by `X-GitHub-Delivery`.
- **Authorisation:** every query is scoped by workspace or user id in SQL; there is no endpoint that takes only a row id.
- **Secrets at rest:** alert targets (Slack URLs) are encrypted with a key derived from a dedicated `DATA_KEY` (AES-256-GCM), not stored in clear.
- **SSRF:** Slack webhooks must match `^https://hooks.slack.com/services/`; no other outbound URLs come from user input.
- **Public routes** (badge, share link): rate limited per IP; share tokens are 192-bit random; responses carry `noindex` and a short cache.
- **Logs:** never log tokens, signatures, Slack URLs or secret values; log delivery id, job id and repo.
- **Code snippets:** secret findings are masked in snippets with the same masker the scanner uses.
- **Deletion:** account deletion also removes installations, triage, settings, shares, alert channels, usage, and cancels the Stripe subscription.

3.8 Target architecture

Everything runs inside the existing Next.js app on Vercel. The only new moving part is a job runner triggered by Vercel Cron, which drains a Postgres queue; no separate server is needed.

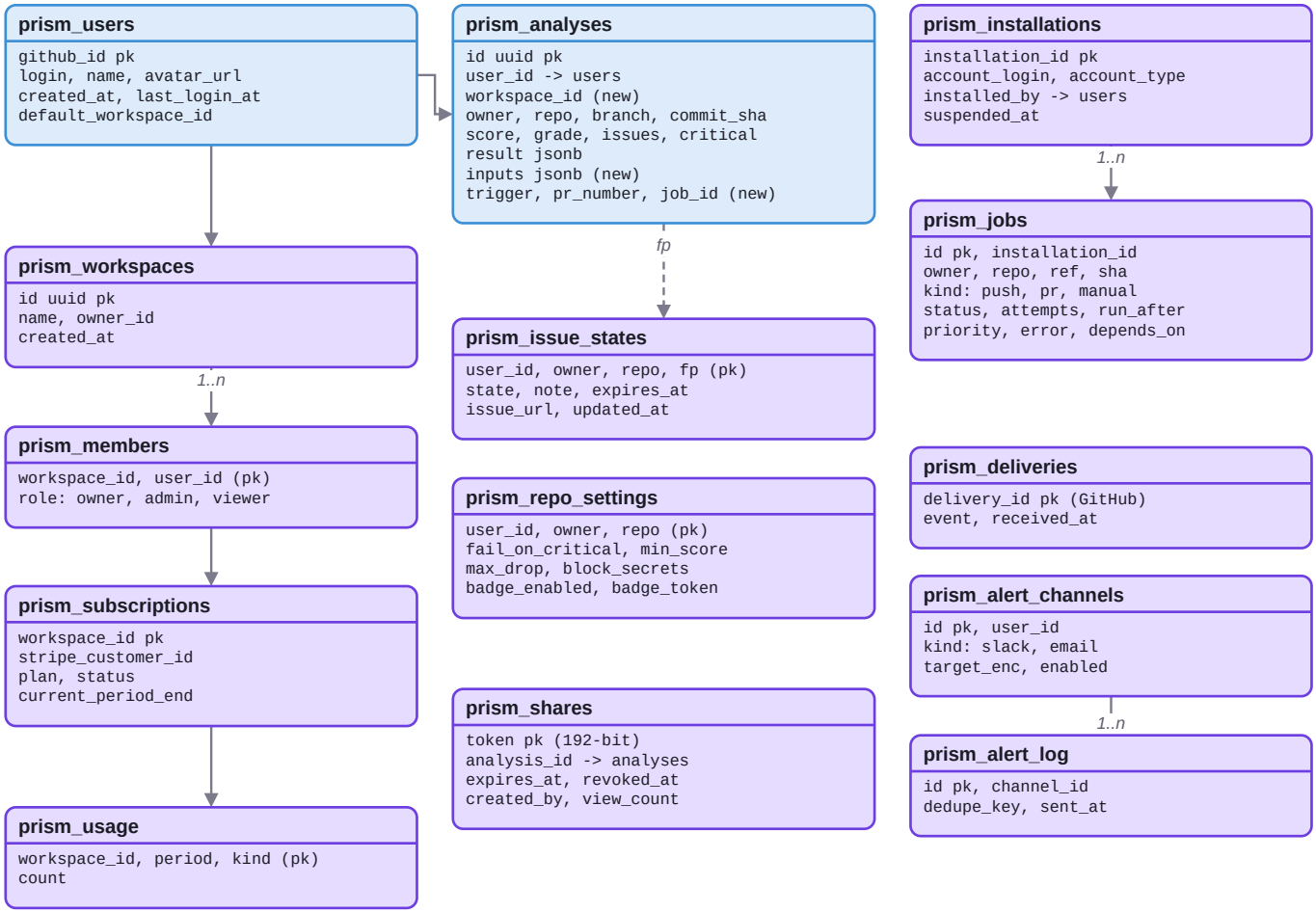


New environment variables

Variable	Purpose	Needed by
`GITHUB_APP_ID`	GitHub App id	F2.1
`GITHUB_APP_PRIVATE_KEY`	PEM used to sign the App JWT (RS256 via jose)	F2.1
`GITHUB_WEBHOOK_SECRET`	HMAC secret for webhook signatures	F2.2
`CRON_SECRET`	Authorises the job-runner route	F2.2
`DATA_KEY`	Encrypts alert targets	F2.6
`RESEND_API_KEY`	Email alerts	F2.6
`STRIPE_SECRET_KEY, STRIPE_WEBHOOK_SECRET, STRIPE_PRICE_PRO, STRIPE_PRICE_TEAM`	Billing	F4.4

4 Data model

New tables are prefixed `prism_` like the existing two. Row level security stays enabled with no policies; only the server connects. Dashed lines are references by value, not foreign keys.



4.1 Changes to existing tables

Table	Change	Why
`prism_analyses`	add `workspace_id uuid, branch_key text, trigger text, pr_number int, job_id uuid, commit_full text, score_version int, inputs jsonb, overflow int`	Webhook and PR runs; diff only on same branch; full sha for APIs; adjusted scores without loading result
`prism_analyses`	unique index becomes `(workspace_id, lower(owner), lower(repo), commit_full, trigger)`	A push and a PR on the same commit are different contexts
`prism_users`	add `default_workspace_id, email, settings jsonb`	Workspaces, alerts

4.2 New tables in detail

Table	Columns	Notes
`prism_issue_states`	user_id, owner, repo, fingerprint (pk); state; note; expires_at; issue_url; updated_at	Becomes workspace-scoped in F4.5
`prism_repo_settings`	workspace_id, owner, repo (pk); fail_on_critical; block_secrets; min_score; max_drop; badge_enabled; badge_token; alert_rules jsonb; auto_analyze	One row per repo; defaults apply when absent
`prism_installations`	installation_id pk; account_login; account_type; installed_by; repo_selection; suspended_at	From the 'installation' webhook
`prism_jobs`	id; installation_id; owner; repo; ref; sha; kind; pr_number; status; priority; attempts; run_after; error; depends_on; created_at; started_at; finished_at	Index on (status, run_after, priority desc, created_at)
`prism_deliveries`	delivery_id pk; event; received_at	Pruned after 7 days
`prism_alert_channels`	id; workspace_id; kind; target_enc; enabled; created_at	Slack URL or email address, encrypted
`prism_alert_log`	id; channel_id; dedupe_key; sent_at	Throttle: one per (channel, dedupe_key) per hour
`prism_shares`	token pk; analysis_id; created_by; expires_at; revoked_at; view_count	Public read-only snapshot
`prism_workspaces / prism_members`	workspace(id, name, owner_id); member(workspace_id, user_id, role)	Role: owner, admin, viewer
`prism_subscriptions / prism_usage`	subscription(workspace_id pk, stripe_customer_id, plan, status, current_period_end); usage(workspace_id, period, kind, count)	Written by the Stripe webhook; usage by features

4.3 Migration plan

- Add `prism_migrations(id text pk, applied_at)` and a runner that applies numbered files in `supabase/migrations/` in a transaction, each exactly once.
- Replace `sql.unsafe(SCHEMA)` on cold start with a check that the ledger is up to date (fail loudly in dev, apply in a deploy step in production).
- Migration 0001 is the current schema as-is, so existing databases adopt the ledger without changes.
- All later changes are additive: new columns nullable first, backfill, then constraints.
- Backfill for old rows: `commit_full` is unknown (set to null; features needing it fall back to the short sha), `inputs` is computed lazily the first time a row is read and written back.

5 Feature specs

Each feature lists the user flow, the rules, the data, the routes and files, the edge cases and the acceptance criteria. IDs (F1.2) are used in the build order. Phase 0 is new: it is the groundwork the other phases need, and it replaces 'tests and CI' at the end of the earlier roadmap.

Phase 0 - Foundations

F0.1 Migrations

Phase 0 - Numbered SQL migrations with a ledger, replacing create-on-first-use.

User flow	- Developer adds `supabase/migrations/0002_issue_states.sql`; the runner applies it once per database. - On deploy, a script runs `npm run db:migrate`; on dev, the first request applies pending files.
Logic and rules	- Ledger table `prism_migrations(id, applied_at)`; files applied in filename order inside a transaction. - A failed migration rolls back and stops the deploy; it never leaves half a schema. - Migration 0001 equals today's schema so existing databases are adopted unchanged. - Advisory lock so two cold starts cannot migrate at once.
Data	- New table `prism_migrations`.
Routes and code	- `src/server/migrate.ts`, `scripts/migrate.ts`, change `db()` in `src/server/db.ts`.
Edge cases	- Database added after users signed in (already handled by upsert in saveAnalysis). - Pooler on port 6543 disallows some session features: use `pg_try_advisory_xact_lock` inside the transaction.
Done when	- Fresh database ends at the same schema as today. - Re-running does nothing. - A broken migration leaves the database unchanged.

F0.2 Fingerprints and richer issues

Phase 0 - Give every issue a stable identity, a rule id and a subject (section 3.1).

User flow	- No user-facing change. History rows start carrying fingerprints, so later features can compare commits.
Logic and rules	- Add `rule, subject, fingerprint` to `Issue` in `analysisTypes.ts`; parsers already know rule and can supply subject. - Occurrence index assigned after sorting findings by (path, line) so it is deterministic. - Keep `id` for display and AI citation; the AI still cites ids, and the fingerprint is looked up from the id. - Record `overflow` = findings dropped by the 300 cap and the per-rule caps.
Data	- Fields live inside `result jsonb`; `inputs jsonb` duplicates the compact form for fast reads.
Routes and code	- `parse.ts`, `secrets.ts`, `deps.ts`, `analysis/index.ts`.
Edge cases	- Two identical lines in one function: occurrence index keeps them apart. - File moved or renamed: all its issues look new and fixed once. Accepted limitation. - Old rows without fingerprints: compare by old id and show 'approximate'.
Done when	- Moving an unrelated block of code above a finding does not change its fingerprint (unit test). - Same repo analysed twice yields identical fingerprints.

F0.3 Score module and stored inputs

Phase 0 - Extract scoring into a pure function; save its inputs (section 3.2).

Logic and rules	- `computeScore(inputs)` returns score and grade; `runAnalysis` calls it. - Saved row gets `score_version` = 1 and `inputs`. - Reading code path: `adjustedScore(inputs, triagedFingerprints)` removes triaged issues from counts and cycles.
Routes and code	- `src/lib/score.ts`, `analysis/index.ts`, `loadRepo.ts`.
Edge cases	- Rows saved before this change have no inputs: compute from `result` on first read and write back.
Done when	- Score for an unchanged repo is identical before and after the refactor (golden test). - Removing one critical issue raises the score by exactly 8 until the cap (property test).

F0.4 Tests and CI

Phase 0 - A test suite for the engine and a pipeline that runs it on every push.

User flow	- Contributor opens a PR; GitHub Actions runs lint, type check, unit tests and a build; the PR is blocked on failure.
Logic and rules	- Vitest for unit and integration tests; fixtures in `tests/fixtures/` are tiny repositories as folders, tarred in memory by a helper. - GitHub and OSV calls are mocked at `fetch`; no test touches the network. - Playwright smoke test: open `/app` (demo data) and the marketing page, no sign-in needed. - Coverage report, not a gate, until the suite stabilises.
Routes and code	- `vitest.config.ts`, `.github/workflows/ci.yml`, `tests/`.
Edge cases	- Parser output depends on the TypeScript version: pin it and snapshot only stable fields.
Done when	- CI is green on main. - Every detector has a positive and a negative test (see section 7).

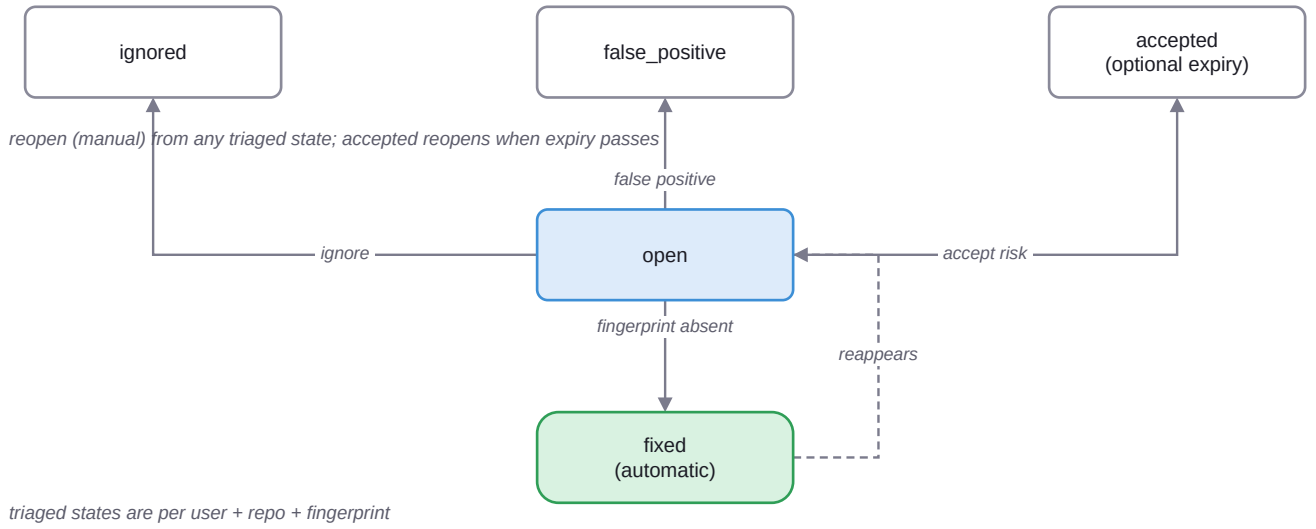
Phase 1 - Make findings actionable

F1.1 Triage findings

Phase 1 - Mark any issue ignored, false positive or accepted; it stops counting and is remembered.

User flow	- On an issue row (Security, Performance, Quality, Overview) the user opens the menu: Ignore, False positive, Accept risk. - A small dialog asks for an optional note; accept risk also offers an expiry (30 days, 90 days, never). - The row moves to the 'Triaged' tab; the score and counts update immediately. - From 'Triaged', Reopen returns it to open.

Logic and rules	- State machine in section 3.3; one row per (user, repo, fingerprint). - Applied at read time in `loadRepo()` after the analysis is loaded: tag each issue with its state; recompute score from inputs. - Accepted risk reopens automatically when `expires_at` passes; the reopen is applied at read time, no job needed. - A fixed fingerprint (absent in the newest analysis) is shown as fixed in the diff but its triage row is kept for 90 days in case it returns. - False positives are tallied per rule; a rule with a high ratio is flagged in the rules table (input for F4.2 and future tuning).
Data	- `prism_issue_states` (section 4.2). Index: `(user_id, lower(owner), lower(repo))`.
Routes and code	- POST `/api/triage` body: owner, repo, fingerprint, state, note, expiresInDays. Server action alternative for the UI. - UI: `SectionView.tsx` row menu, Triaged tab; `loadRepo.ts` overlay; `db.ts` helpers.
Edge cases	- Signed out: menu hidden; tooltip says sign in to triage. - Public repo, many viewers: triage is per viewer, never shared. - Note length capped at 500 characters; stored as text, rendered escaped. - Triage on an issue that is also in the AI review: the review text stays, a 'triaged' chip is shown.
Done when	- Ignoring a critical issue raises the displayed score by 8 (within caps) and survives a new commit that moves its line. - Reopen restores the original score. - Another user's triage never affects mine.

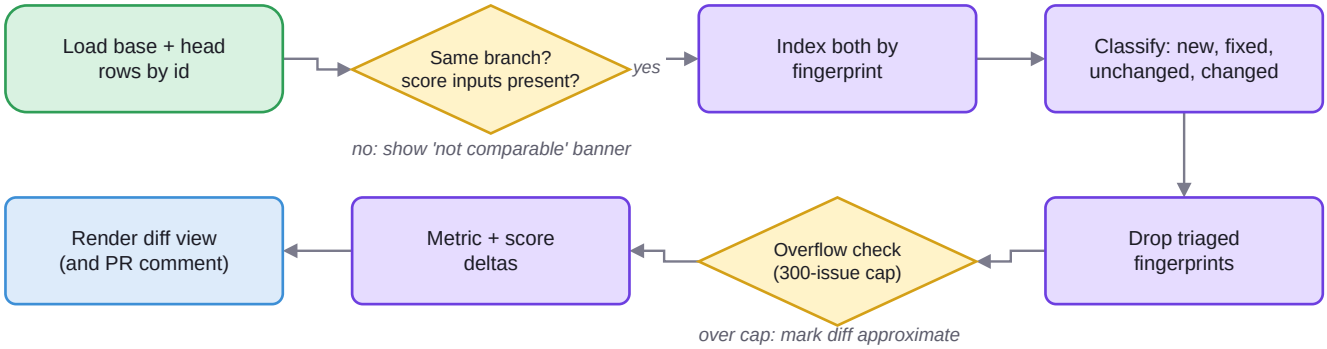


F1.2 Diff between analyses

Phase 1 - Compare two analyses of the same repo: new, fixed, unchanged and changed issues, with metric and score deltas.

User flow	- History page: pick any two rows (default: latest versus previous on the same branch), or press Compare on a row. - The diff view shows score change, then tabs: New, Fixed, Changed, Unchanged; and a metric table (complexity, duplication, cycles, dependencies). - Each issue links to its detail; New issues can be triaged inline.

Logic and rules	- Algorithm in the diagram below. Index both result sets by fingerprint. - Classify: in head only = new; in base only = fixed; in both = unchanged, or changed when severity, risk or category differ. - Exclude triaged fingerprints from New and from the counts, but list them in a collapsed 'Triaged' group. - Same branch required for the default comparison; cross-branch is allowed with a banner. - If either side lacks fingerprints (old rows) compare by old id and show 'approximate'. - If `overflow > 0` on either side, show 'approximate: some lower-ranked issues are not listed' (gap G3).
Data	- No new tables: reads two rows from `prism_analyses` by id, scoped to the user or workspace.
Routes and code	- `/app/r/[owner]/[repo]/diff?base=ID&head=ID`, `src/lib/diff.ts` (pure, shared with PR comments), `HistoryView.tsx`.
Edge cases	- First analysis: nothing to compare; show 'first analysis'. - Base row belongs to another user: 404. - Score version differs: banner 'scoring changed between these runs'. - Rows more than 200 issues: diff is computed on the server, not in the browser.
Done when	- Fixing one issue and adding one produces exactly one Fixed and one New. - Diff of a row with itself is empty. - Moving code without changing it produces an empty diff (fingerprint test).



F1.3 Score trend

Phase 1 - A chart of score over time per repository, with critical counts and event markers.

User flow	- Overview and History show a line chart: x = analysis time, y = adjusted score; hovering shows commit, score, issues, critical. - Toggle: adjusted (default) or raw; range: 10 runs, 30 days, all.
Logic and rules	- One point per (branch, commit); the default branch only unless the user picks another. - Adjusted score from stored inputs and current triage (section 3.2); raw from the `score` column. - Markers: a dot where a critical issue first appeared, a flag where scoring version changed. - Chart is inline SVG, no chart library; accessible: a table of the same points is available under the chart.
Data	- Query: `select id, commit_sha, created_at, score, inputs from prism_analyses where ... order by created_at desc limit 50`.
Routes and code	- `src/components/app/overview/Widgets.tsx` (replace the sparkline), `HistoryView.tsx`, `repoHistory(limit)` raised from 12 to 50.
Edge cases	- One data point: show the value and 'run again later to see a trend'. - Retention window of the plan hides older points. - Analyses for the same commit on two triggers: keep the most recent.

Done when	- Chart matches the table of points exactly. - Toggle changes only the line, not the axis labels.

F1.4 Code snippet view

Phase 1 - Show the offending lines, with context, next to each finding.

User flow	- Expanding a finding loads about 8 lines before and after, with the flagged line highlighted and a link 'Open on GitHub at this commit'.
Logic and rules	- Fetch on demand: GitHub contents API for `path` at `commit_full` (never the moving branch), using the viewer's token for private repos or the server token for public. - Mask secrets: run the scanner's masker over the text before returning it; for secret findings show the line with the value replaced by its masked form. - Cache by (owner, repo, sha, path) for one hour; snippets are not stored in the database. - Only fetch for issues with a line number; file-level findings (large file, cycle) link to the file instead. - Respect the 400 KB per file limit; larger files show a link only.
Data	- None stored. Needs `commit_full` (F0.2 and the column in 4.1).
Routes and code	- GET `/api/snippet?owner&repo&sha&path&line` (session required for private), UI in `SectionView.tsx`.
Edge cases	- File deleted or sha garbage-collected: 'This file is no longer available'. - GitHub rate limit: show the issue without a snippet and a retry link. - Binary or minified file: no snippet. - Line ending and tab handling: normalise to LF, keep tabs, render with a fixed tab width.
Done when	- The highlighted line is exactly the line in the finding. - A secret value never appears in the response body (test on every secret rule).

Phase 2 - Make it continuous

F2.1 GitHub App and installation

Phase 2 - Let people install PRISM on their repositories so it can read code and write checks without a personal token.

User flow	- Settings, Repositories, 'Connect GitHub' sends the user to the App installation page; they choose all repositories or selected ones. - GitHub returns to `/app/setup/github?installation_id=...`; PRISM links the installation to the signed-in user. - The user sees the repositories and toggles automatic analysis per repo.
Logic and rules	- Register a GitHub App with permissions: contents read, metadata read, pull requests write, checks write, issues write (F2.7), and subscribe to push, pull_request, installation and installation_repositories events. Contents write is NOT requested here; it is requested later only for AI fix PRs (F4.3). - User sign-in can stay on the existing OAuth app; the App is for repository access. An installation is attached to a workspace by the user who installed it. - Installation token flow: sign a JWT (RS256, 9 minutes, claims iss = App id) with `jose`, POST `/app/installations/ID/access_tokens`, cache the token for 50 minutes in memory. - Installation webhook events create, suspend or remove rows; removed installations stop all jobs and show a banner.
Data	- `prism_installations`, `prism_repo_settings.auto_analyze`.

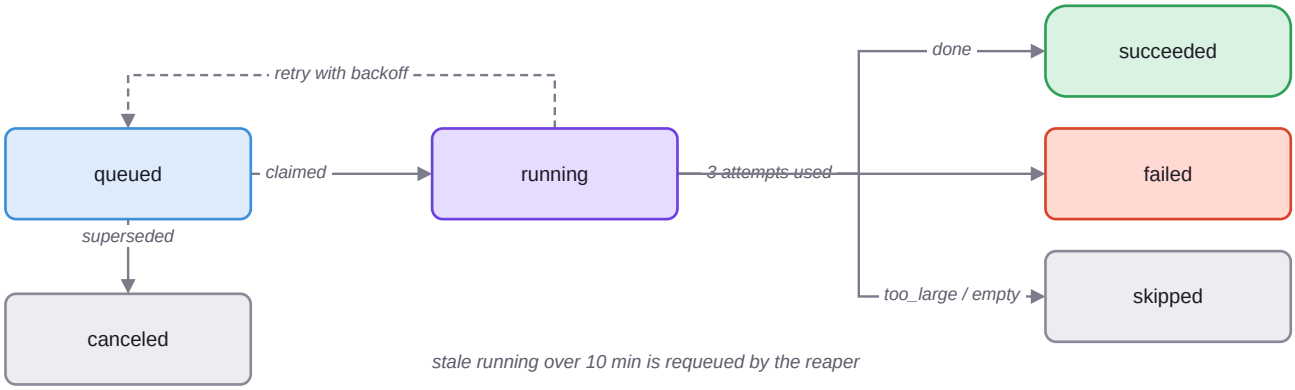
Routes and code	- `src/server/githubApp.ts`, `/app/setup/github`, `/api/github/webhook` (installation events).
Edge cases	- Installation by someone who is not a PRISM user: show a 'sign in to finish' page. - Repository removed from the installation: disable its auto analysis, keep history. - Private key rotation: support two keys during rollover. - User revokes the App on GitHub: the next token request fails with 401; mark suspended and notify.
Done when	- After installing, a push to a selected repo creates a job within 5 seconds. - Uninstalling stops jobs and keeps saved history.

F2.2 Webhooks and job queue

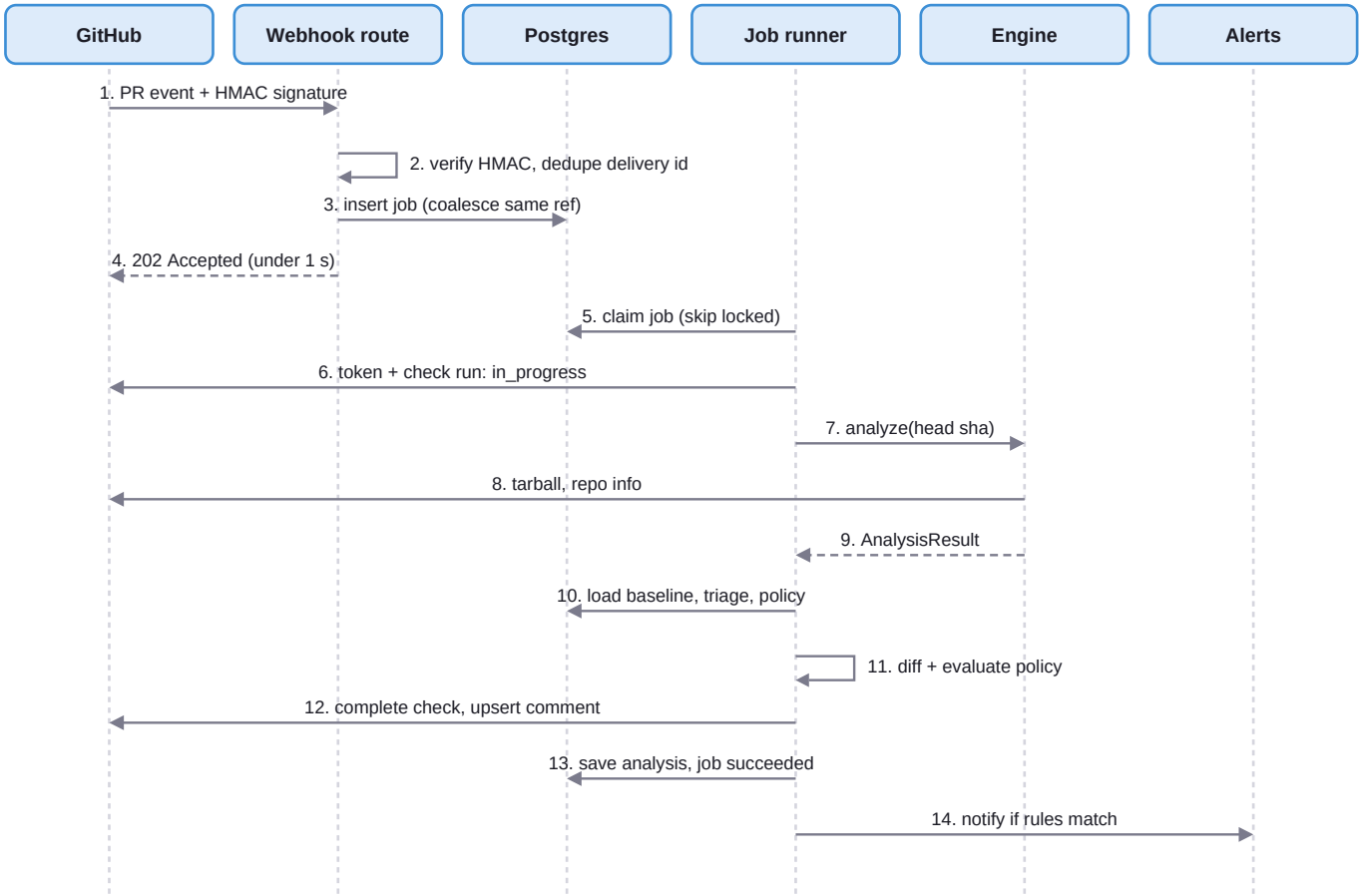
Phase 2 - Receive GitHub events reliably and run analyses in the background.

User flow	- No user action. The status appears in Setup (last delivery, queue depth, last failure) and in History as run rows.
Logic and rules	- Route verifies the signature on the raw body, checks the delivery id against `prism_deliveries` (insert; ignore if present), enqueues, and returns 202 within a second. It never analyses inline. - Which events enqueue: `push` to the default branch; `pull_request` opened, synchronize, reopened, ready_for_review. Ignored: drafts (unless enabled), forks without access, bot branches, repos with auto analysis off. - Coalescing: before inserting, mark older queued jobs for the same (repo, ref) as canceled, so ten quick pushes cost one analysis. - Runner: Vercel Cron calls `/api/jobs/run` every minute with `CRON_SECRET`; it claims up to 2 jobs with `for update skip locked`, priority first (PR before push, interactive before background). - Per job: get token, create check run, analyse head, load baseline, evaluate policy, update check and comment, save, alert. Failure handling follows section 3.6; retries back off 1, 5, 15 minutes, max 3 attempts. - A reaper at the start of each run requeues jobs stuck in running for over 10 minutes and prunes deliveries older than 7 days. - Baseline: for a PR, the analysis of the base sha; if missing, enqueue it first (`depends_on`) with priority; if it still fails, run with no baseline (policy rule 6).
Data	- `prism_jobs`, `prism_deliveries`.
Routes and code	- `/api/github/webhook`, `/api/jobs/run`, `src/server/jobs.ts`, `vercel.json` crons.
Edge cases	- GitHub redelivers on timeout: dedupe by delivery id. - Webhook arrives before the installation row exists: enqueue anyway; the runner fetches installation info lazily. - Two runners overlap: skip locked prevents double claims. - Function timeout: `maxDuration` 300; the analysis has its own 90 second budget and fails as 'network' if exceeded. - Burst of 100 repos pushing at once: queue depth grows; the oldest background jobs wait, PR jobs jump the queue.
Done when	- A forged signature gets 401 and creates nothing. - Redelivering the same event twice creates one job. - Killing the runner mid-job results in the job running again within 10 minutes and exactly one saved analysis.

Job lifecycle



Pull request run, end to end



F2.3 PR check and comment

Phase 2 - Report, on the pull request itself, what the change did to health.

User flow	- The developer opens a PR. A check named 'PRISM' appears as in progress, then completes with a title such as 'Health 82 to 76 (-6): 2 new critical'. - A single comment on the PR shows a summary table and the new issues; it is updated on every push, never duplicated. - Annotations mark the changed lines of new issues in the Files tab.
Logic and rules	- Policy from section 3.4 decides success, neutral or failure; only new, untriaged issues count. - Check output: title, a summary (score before and after, counts by severity, top 5 new issues with ids), and up to 50 annotations (GitHub's per-request limit; send further batches by updating the check). - Annotation level: critical and high = failure, medium = warning; path and line from the issue location; message includes the issue id and a link to the PRISM issue. - Comment is identified by a hidden marker `PRISM-REPORT` in an HTML comment; update by finding it in the PR's comments, else create it. - Comment includes a triage hint: 'Reply or use the PRISM app to ignore an issue'. v1 triage happens in the app only. - Required status: users decide in branch protection whether the check blocks merge; PRISM only reports.
Data	- Reads baseline and head from `prism_analyses`; stores `trigger = 'pr'` and `pr_number`.
Routes and code	- `src/server/githubChecks.ts`, `src/server/prComment.ts`, `src/lib/diff.ts` and `src/lib/policy.ts`.
Edge cases	- Force-push after a run: the new sha makes a new check; the older run is stale and its check is left as is. - PR from a fork: GitHub gives read-only tokens; analyse the head via the base repo's installation if the fork's content is reachable, otherwise post a neutral check 'cannot read fork'. - Very large PR: analysis is of the whole head tree, not just changed files; the comment lists only issues whose fingerprints are new. - Comment edit conflicts: last writer wins; the marker keeps it to one comment. - Rate limit on checks API: retry with the Retry-After header.
Done when	- A PR that adds one eval shows one new critical and a failing check; removing it passes the check on the next push. - Ten pushes keep one comment. - Triaged issues never appear as new.

F2.4 Repo settings and policy

Phase 2 - Per-repository controls for automatic analysis, check strictness, badge and alerts.

User flow	- Settings, Repositories lists connected repos; clicking one shows toggles and thresholds with a live preview: 'With these settings, the last 10 PRs would have: 7 pass, 2 warn, 1 fail'.
Logic and rules	- Defaults when no row exists: auto analysis off; fail on critical on; block secrets on; min score off; max drop off; badge off. - Settings are validated server-side (score 0 to 100, drop 1 to 50). - Preview replays policy over stored PR analyses; it does not call GitHub.
Data	- `prism_repo_settings`.
Routes and code	- `/app/settings/repos/[owner]/[repo]`, `src/lib/policy.ts` (shared).
Edge cases	- Owner without admin rights on a team workspace: read-only view. - Changing thresholds does not rewrite past checks.

Done when	- Setting min score 90 makes the next run on a score-85 PR fail with that reason named in the check.

F2.5 README badge

Phase 2 - An SVG badge showing the current score for any repo, for the README.

User flow	- Repository settings shows the Markdown to copy: `![PRISM](https://APP/api/badge/owner/repo.svg)`; private repos use a token URL.
Logic and rules	- Public repos: badge works for anyone, using the most recent analysis of the default branch (any user's, the shared public result if none saved). - Private repos: disabled by default; when enabled the URL carries a random `badge_token` so the score does not leak; token can be rotated. - Colour from grade: A and B bright green, C yellow, D and F red, unknown grey 'not analysed'. Text is 'PRISM 82 B+'. - SVG rendered from a template string, no external fonts or requests; `Cache-Control: public, s-maxage=300, stale-while-revalidate=86400`. - Rate limited per IP; unknown repos return the grey badge with a 200 so READMEs never show a broken image.
Data	- `prism_repo_settings.badge_enabled, badge_token`.
Routes and code	- `/api/badge/[owner]/[repo]/route.ts`.
Edge cases	- Name with dots or case differences: normalise to lower case. - Analysis in progress: serve the last known score. - Rotating the token breaks old URLs on purpose.
Done when	- Badge for a repo scores the same as its dashboard (adjusted for the owner, raw for everyone else). - No badge request ever reads a private repo without the token.

F2.6 Alerts

Phase 2 - Tell people on Slack or email when something important happens.

User flow	- Settings, Alerts: add a channel (paste a Slack incoming webhook URL, or confirm an email address via a link), choose events, send a test message.
Logic and rules	- Events: new critical issue; new secret; score drops by N or more; scheduled analysis failed 3 times; installation suspended. - Evaluated after each saved analysis against the previous analysis on the same branch, untriaged only. - Dedupe key = event + repo + fingerprint (or sha for score drops); the same key is never sent twice to a channel; also one message per channel per repo per hour, extra events are folded into a digest line. - Slack: Block Kit message with score before and after, up to 3 issues, button to PRISM. Email: plain and HTML versions through Resend. - Targets are encrypted with `DATA_KEY`; the UI shows only a masked suffix. - Failed sends retry twice; three consecutive failures disable the channel and tell the owner in the app.
Data	- `prism_alert_channels, prism_alert_log`, rules in `prism_repo_settings.alert_rules`.
Routes and code	- `src/server/alerts.ts`, `/app/settings/alerts`, `/api/alerts/test`.

Edge cases	- Slack URL revoked (404): disable channel, notify in app. - Email bounce: disable and say so. - Quiet hours optional later; not in v1. - Free plan: alerts UI shows the upgrade prompt, nothing is sent.
Done when	- A new committed key on a watched repo posts to Slack within two minutes of the push. - Replaying the same webhook sends no second message.

F2.7 Create GitHub issue from a finding

Phase 2 - Turn a finding into a tracked GitHub issue without leaving PRISM.

User flow	- On a finding: Create issue. A dialog previews title and body, lets the user choose labels and assignees (from the repo), and creates it. - The finding then shows 'Tracked: #123' and links to it.
Logic and rules	- Title: `[PRISM]` finding title; body: severity, location with a permalink to the commit, impact, the AI explanation and fix if a review exists, fingerprint and PRISM link. - Duplicate guard: if `issue_url` already exists for this fingerprint, link to it instead of creating another. - Auto resolve: when a later analysis no longer has the fingerprint, comment 'Fixed in SHA' on the issue; closing it is an opt-in setting. - Needs the App's issues write permission; if missing, show how to grant it.
Data	- `prism_issue_states.issue_url`.
Routes and code	- POST `/api/issues`, `src/server/githubIssues.ts`.
Edge cases	- Issues disabled on the repo: explain and offer copying the text. - Repo moved or renamed: GitHub redirects; store the final URL. - Secret findings: body contains the masked form and instructions to rotate the key, never the value.
Done when	- Clicking twice creates one issue. - The permalink opens the exact line on the analysed commit.

Phase 3 - Use the data

F3.1 Exports and SARIF

Phase 3 - Take results out of PRISM: Markdown, JSON, CSV, a printable report that saves as PDF, and SARIF for GitHub code scanning.

User flow	- Report toolbar: Export. Choose format; the browser downloads, or opens the print view for PDF. - SARIF has a second action: 'Send to GitHub Security tab' (needs security events write).
Logic and rules	- Markdown and CSV: columns id, severity, category, rule, title, location, impact, risk, state. - JSON: the stored `AnalysisResult` plus triage states. - Printable report: a route with a print stylesheet (cover, score, top risks, hotspots, full issue table); user prints to PDF. No PDF library, so no server cost. - SARIF 2.1.0 mapping: tool.driver = PRISM; rules from rule ids; results with level (critical, high = error; medium = warning), message, `physicalLocation` (uri relative to repo root, `startLine`), and `partialFingerprints.primaryLocationLineHash = fingerprint` so GitHub tracks alerts across commits. - Upload: gzip then base64 the SARIF and POST to `/repos/OWNER/REPO/code-scanning/sarifs` with commit sha, ref and the checkout URI.

Routes and code	- `/api/export/[owner]/[repo]?format=md|json|csv|sarif&id=`, `/app/r/[owner]/[repo]/report` (print view), `src/lib/sarif.ts`.
Edge cases	- Secrets: values are masked in every format. - SARIF over 10 MB or 25,000 results: truncate to top risk and say so in a note. - Triaged issues: exported with their state; SARIF marks them with `suppressions`.
Done when	- The SARIF validates against the official schema in a test. - The Security tab shows the same issues as PRISM after upload.

F3.2 Churn x complexity hotspots

Phase 3 - Rank files that are both complex and frequently changed, which is where bugs concentrate.

User flow	- Overview and Architecture show hotspots with a churn column (commits in the last 90 days); the 3D map can colour by churn.
Logic and rules	- The tarball has no history, so ask GitHub GraphQL for the top 30 files by complexity plus fan-in: one query with 30 aliased `history(path, since)` fields returning `totalCount`. - New file risk (version 2): `complexity share * 35 + issue weight (max 35) + fan-in share * 12 + churn share * 18`, capped at 99. Churn share = file commits divided by the highest commit count among queried files. - File risk affects hotspots and the map, not the health score; scores stay comparable across the change. - Without any token (no user token, no server token) skip with the existing note: churn needs a token.
Data	- `hotspots[].churn` in the result; `riskVersion = 2`.
Routes and code	- `src/server/analysis/churn.ts`, `analysis/index.ts`.
Edge cases	- Squash-merged repos: low commit counts per file; the share still ranks correctly. - GraphQL cost: 30 aliases is one call; fine within limits. - Generated files with huge churn: honour the non-production path filter.
Done when	- A file with many commits and high complexity outranks a complex but untouched file.

F3.3 Compare repositories and branches

Phase 3 - Side-by-side comparison of up to 4 repositories or two branches of one.

User flow	- Compare page: pick repos from the user's analysed list; see score, grade, issues by severity, quality metrics and top risks in columns; differences highlighted.
Logic and rules	- Uses the most recent analysis per repo (default branch), adjusted by the viewer's triage. - Branches: analyse each (manual or by job) and diff with F1.2. - Share of totals normalised by file count where useful ('critical per 100 files') to avoid unfair comparison of large and small repos.
Routes and code	- `/app/compare?repos=a/b,c/d`, `userLatestPerRepo()` query.
Edge cases	- Different scoring versions: banner. Repos never analysed: prompt to analyse.
Done when	- The columns match each repo's own dashboard numbers.

F3.4 Organisation dashboard

Phase 3 - One page that answers 'which of my repositories need attention?'

User flow	- Repositories page shows a table: repo, score, grade, trend sparkline, critical, last analysed, auto-analysis status; sortable; filters for stale and failing.
Logic and rules	- Latest analysis per repo via `select distinct on (lower(owner), lower(repo))` ordered by created_at. - Aggregates: average score, total critical, repos below threshold, repos not analysed in 30 days. - Worst offenders list for the week: biggest score drops across all repos.
Routes and code	- `/app/repos` (extend).
Edge cases	- Hundreds of repos: paginate at 50; aggregates computed in SQL.
Done when	- Totals equal the sum over rows; clicking a row opens that repo.

Phase 4 - Product

F4.1 More languages and a real Python parser

Phase 4 - Add Go, Java and Rust and parse Python properly.

Logic and rules	- Language interface: `{ extensions, parse(path, text) -> FileFacts }` registered in `languages/index.ts`; the engine loops over registered parsers. - Parsing via web-tree-sitter with prebuilt WASM grammars (works in serverless, no native build); cyclomatic complexity computed from branch node types per grammar. - Starter rules: Go (string-built SQL, `exec.Command` with concatenation, ignored errors), Java (string-built SQL in Statement, `Runtime.exec`), Rust (`unsafe` blocks, `unwrap` in non-test code); Python moves from line scanning to tree-sitter for functions and complexity. - Imports resolved per language: Go module path, Java package, Rust `mod` and `use crate`; unresolved imports are external. - Languages not parsed are still counted in the language bar.
Routes and code	- `src/server/analysis/languages/*.ts`, `parse.ts` becomes the JS/TS entry.
Edge cases	- Grammar load adds cold start time: lazy-load per language. - Generated code (protobuf, bindata): skip by header comment and path patterns.
Done when	- Each language has fixtures and tests for each rule (positive and negative). - Adding Python AST does not change scores on the existing fixtures by more than 2 points; differences are reviewed.

F4.2 Ask your repo

Phase 4 - A chat on the AI Review page that answers questions about the analysis, with citations.

User flow	- The user types 'what should I fix first?' or 'why is auth/session.ts risky?'; the answer streams in with issue chips that open the finding.

Logic and rules	- Server tools the model can call, all read-only on the stored analysis: `list_issues(filter)`, `get_issue(id)`, `get_file_facts(path)`, `get_diff(base, head)`, `get_hotspots()`. - System prompt keeps today's rules: use only facts, cite ids, no invented files or numbers, say when static rules may have misfired. - No source code is sent unless the user presses 'include this snippet' on a finding (then only that snippet, masked). - Conversation kept in the browser (last 10 turns); nothing stored server-side in v1. - Quota from the plan (section 3.5); usage counted per message; over quota shows the reset time. - Model call uses the same client as `ai.ts` with the server-side fallback beta; streaming via the SDK.
Routes and code	- POST `/api/chat` (stream), `src/server/chat.ts`, `AiReviewView.tsx`.
Edge cases	- Prompt injection through repo text (issue titles come from code): treat all tool output as data; tools are read-only so the worst case is a wrong answer. - Empty analysis: say so instead of guessing. - Model unavailable: show 'AI is unavailable' and the plain facts.
Done when	- Every answer that names a problem includes at least one valid issue id. - An answer about a file not in the analysis says it is not in the analysis.

F4.3 AI fix pull requests

Phase 4 - Ask for a proposed fix for a finding and open it as a reviewable pull request.

User flow	- On a finding: Propose fix. The user is told that this snippet will be sent to the AI; they confirm. A diff preview appears with the explanation and false-positive risk; Open PR creates it.
Logic and rules	- Opt-in per repository ('Allow sending code snippets to AI'); requires the App's contents write permission, requested only now. - Input to the model: the finding facts plus the masked snippet (about 40 lines) and the file's language; output: a unified diff touching only that file, an explanation and a risk note. - Validation before showing: the diff applies cleanly to the file at the analysed sha; the patched file parses with the same parser; re-running the single rule on the patched text no longer fires; otherwise the proposal is rejected and the user sees why. - PR creation through the Git data API: branch `prism/fix-` from the analysed sha, one commit, PR body with the explanation, fingerprint and 'generated by PRISM'. Never auto-merge. - Monthly quota by plan.
Data	- Tracked state in `prism_issue_states` with the PR URL.
Routes and code	- POST `/api/fix`, `src/server/fixPr.ts`.
Edge cases	- Secrets: no fix PR; rotate-the-key guidance instead. - Default branch moved on: the patch is applied to the sha; GitHub shows conflicts if any. - Repo forbids branch creation: show the diff for manual apply.
Done when	- A proposed fix that does not apply is never offered. - The opened PR passes PRISM's own check for that finding.

F4.4 Plans and billing

Phase 4 - Free and paid plans enforced by one function, paid through Stripe.

User flow	- Pricing page; Upgrade opens Stripe Checkout; the Customer Portal handles cards, invoices and cancellation; usage and limits are visible in Settings, Billing.
Logic and rules	- Entitlements table in section 3.5; features call `can()`, which reads the subscription (cached for a minute) and the usage counters. - Stripe webhook events: `checkout.session.completed, customer.subscription.updated, customer.subscription.deleted, invoice.payment_failed`; verified with the signing secret; idempotent by event id. - Statuses: active and trialing grant the plan; past_due keeps it for 7 days with a banner; canceled returns to Free at period end. - Counters reset monthly (`period = YYYY-MM`); counters are incremented before the paid action and refunded on failure.
Data	- `prism_subscriptions, prism_usage`.
Routes and code	- POST `/api/billing/checkout`, `/api/billing/portal`, `/api/stripe/webhook`, `src/server/plans.ts`.
Edge cases	- Webhook arrives before the user returns from Checkout: both paths converge on the same row. - Refund or dispute: treated as cancel. - Tax and invoices: delegated to Stripe Tax and the portal.
Done when	- A Free user hitting the private repo limit sees which limit and how to raise it. - Replaying a Stripe event changes nothing.

F4.5 Workspaces and teams

Phase 4 - Group repositories and people so results belong to a team, not one person.

User flow	- Every user has a personal workspace. Create a team workspace, invite by GitHub login, set roles; switch workspace from the sidebar.
Logic and rules	- Roles: owner (billing, delete), admin (settings, triage, repos), viewer (read). - Analyses, triage, settings, installations, alert channels and subscriptions move from user to workspace scope; migration copies each user's data into their personal workspace. - Invitations are links with a 7 day expiry bound to a GitHub login; accepted by signing in as that login. - Private repo analyses visible to the workspace only if the installing user's installation covers them (GitHub is the source of truth for access).
Data	- `prism_workspaces, prism_members` plus `workspace_id` on scoped tables.
Routes and code	- `/app/settings/team`, `src/server/workspace.ts` resolves the active workspace for every query.
Edge cases	- Last owner leaves: blocked. - Member removed from the GitHub org: loses private repo visibility at the next token check.
Done when	- A viewer cannot change settings (server enforced). - Moving to workspaces changes no existing user's numbers.

F4.6 Share links

Phase 4 - Send a read-only snapshot of a report to someone who is not a PRISM user.

User flow	- Report toolbar: Share. Choose expiry (24 hours, 7 days, 30 days); a warning appears for private repositories; copy the link. Active links are listed with Revoke.
Logic and rules	- A share points at one saved analysis (a snapshot), not live data; token is 24 random bytes, base64url. - The public page renders the report without sign-in, hides the sidebar and actions, shows 'Shared by LOGIN, expires DATE', sets `noindex`, and counts views. - Private repo shares hide file paths until the creator ticks 'show file names' in the dialog. - Revoked or expired: 404 page with no detail.
Data	- `prism_shares`.
Routes and code	- `/s/[token]`, `/api/shares`.
Edge cases	- Creator deletes the account: shares are deleted with it. - Plan limit on active links applies at creation.
Done when	- Revoking makes the link fail within a minute (cache TTL). - A share never exposes another analysis.

F4.7 Settings, onboarding and error states

Phase 4 - The surface that makes the app feel finished.

User flow	- Settings: Profile, GitHub access, Repositories, Alerts, Billing, Team, Data. - First run: three steps (connect GitHub, choose repos, run first analysis) with a progress bar; skippable. - Empty states for each section say what will appear and the one action that makes it appear.
Logic and rules	- Data page: export everything (JSON), delete account (existing endpoint, extended in 3.7). - Failed runs show in History as red rows with the error and a Retry button (re-enqueues). - A global banner reports degraded services (database, GitHub rate limit, AI) from the same checks as `/app/setup`. - Rate limit errors show the reset time from GitHub's headers.
Routes and code	- `/app/settings/*`, `RepoError.tsx`, `SetupClient.tsx`.
Edge cases	- JavaScript disabled: server-rendered pages keep working for reading.
Done when	- A new user reaches a first analysis in under three minutes without documentation.

6 Routes and screens

Existing routes are unchanged. New ones are listed with who can reach them.

Route	Type	Who	Feature
`/app/r/[owner]/[repo]/diff`	Page	Signed in (own data)	F1.2
`/app/r/[owner]/[repo]/report`	Page (print)	Signed in	F3.1
`/app/settings/*`	Pages	Signed in	F2.4, F2.6, F4.4, F4.5, F4.7
`/app/compare`	Page	Signed in	F3.3
`/app/setup/github`	Page	Signed in	F2.1
`/s/[token]`	Page	Public (token)	F4.6
`/api/triage`	POST	Signed in	F1.1
`/api/snippet`	GET	Signed in or public repo	F1.4
`/api/github/webhook`	POST	GitHub (signature)	F2.1, F2.2
`/api/jobs/run`	GET/POST	Cron (secret)	F2.2
`/api/badge/[owner]/[repo]`	GET svg	Public (token for private)	F2.5
`/api/alerts/test`	POST	Signed in	F2.6
`/api/issues`	POST	Signed in	F2.7
`/api/export/[owner]/[repo]`	GET	Signed in	F3.1
`/api/chat`	POST stream	Signed in	F4.2
`/api/fix`	POST	Signed in	F4.3
`/api/billing/*, /api/stripe/webhook`	POST	Signed in / Stripe	F4.4
`/api/shares`	POST/DELETE	Signed in	F4.6

Sidebar after this spec

Overview, Repository, Architecture, Security, Performance, Quality, AI Review (with chat), History (with diff and trend), Compare, and a Settings entry in the account menu. Repositories (all repos) stays reachable from the top bar.

7 Test plan and CI

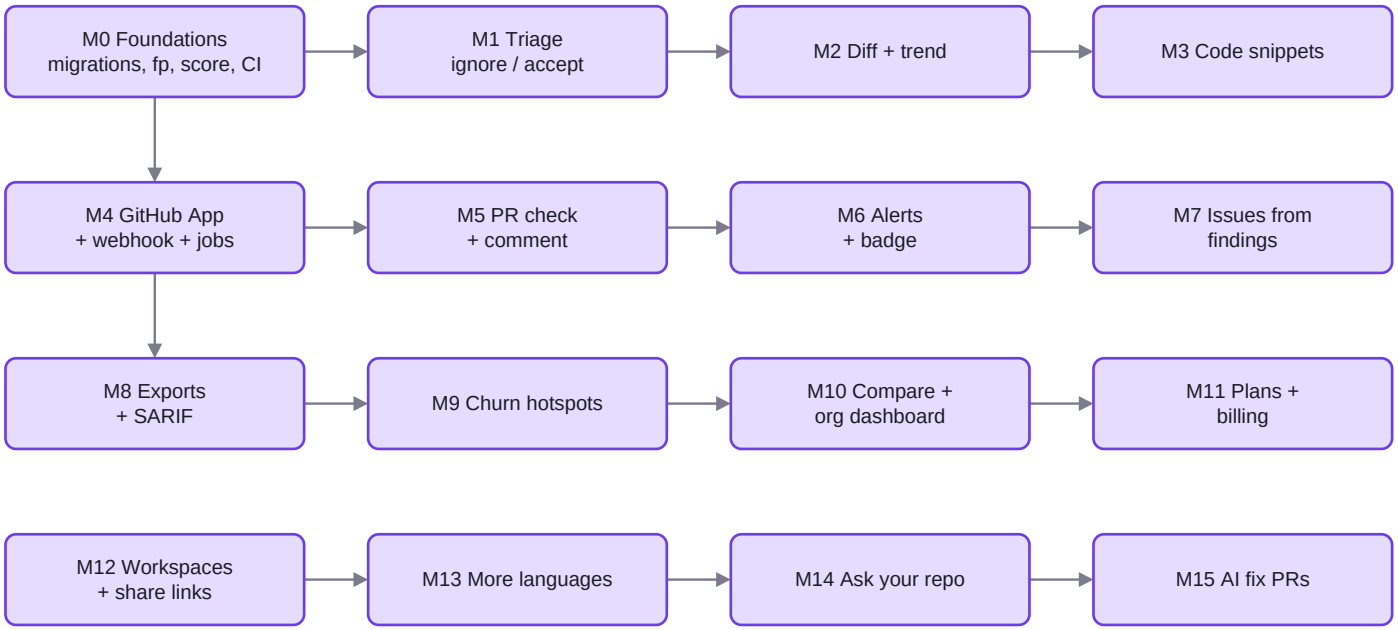
Area	Test	Type
Each rule in parse.ts	Positive fixture fires; benign lookalike does not (for example `eval` in a string)	Unit
secrets.ts	Every provider pattern matches; values never appear in output; test keys in fixtures are obviously fake	Unit
graph.ts	Cycle of 2 and 3 found; self-import ignored; fan-in correct	Unit
Fingerprints	Stable across line moves; differ across files; occurrence index deterministic	Unit
Score	Golden values; monotonic in each input; caps hold; adjusted score equals recompute	Unit and property
Diff	New, fixed, unchanged, changed; triage exclusion; overflow flag; old rows approximate	Unit
Policy	Each of the 7 rules in order; first match wins	Unit
Webhook	Valid, invalid and missing signature; duplicate delivery; coalescing; ignored events	Integration
Jobs	Claim concurrency with two runners; retry backoff; reaper requeues stale; priority order	Integration (real Postgres in CI)
SARIF	Validates against the schema; fingerprints present	Unit
Badge	Colours by grade; unknown repo grey; private needs token	Unit
Alerts	Dedupe key; hourly throttle; encryption round trip	Unit
Plans	Every limit in the table, upgrade and downgrade	Unit
Smoke	Marketing page and demo dashboard load; no console errors	Playwright

CI workflow

- On push and pull request: install, `npm run lint`, `tsc --noEmit`, `vitest run`, `next build`.
- A Postgres service container runs migrations from scratch, then the integration tests.
- Playwright smoke runs against the built app.
- PRISM analyses itself on pull requests once F2.3 ships (dogfooding).

8 Build order and milestones

Phases group the work by outcome; milestones are the shippable steps. Each milestone ends with something a user can see or a risk that is removed.



Order is the recommended sequence. M2 must land before M5 (PR comments reuse the diff); M4 before everything automatic.

Milestone	Contains	Depends on	Ships when
M0	F0.1 to F0.4	None	CI green, fingerprints and score module in place, no visible change
M1	F1.1 triage	M0	Ignore an issue and see the score change
M2	F1.2 diff, F1.3 trend	M0, M1	History shows a diff and a chart
M3	F1.4 snippets	M0	Findings show code
M4	F2.1 App, F2.2 webhooks and jobs	M0	A push creates a saved analysis automatically
M5	F2.3 PR check and comment, F2.4 settings	M2, M4	A PR gets a check and one comment
M6	F2.5 badge, F2.6 alerts	M4	Badge in a README; Slack message on a new critical
M7	F2.7 issues from findings	M4	One click creates a GitHub issue
M8	F3.1 exports and SARIF	M0	Security tab shows PRISM alerts
M9	F3.2 churn	M0	Hotspots reflect change frequency
M10	F3.3 compare, F3.4 org dashboard	M2	One page for all repos
M11	F4.4 plans and billing	M5	Limits enforced, payments work
M12	F4.5 workspaces, F4.6 share links	M11	Teams and shareable reports
M13	F4.1 languages	M0	Go, Java, Rust analysed
M14	F4.2 ask your repo	M2	Chat answers with citations

Milestone	Contains	Depends on	Ships when
M15	F4.3 AI fix PRs	M7, M14	A fix PR passes its own check

8.1 Release checklist for every milestone

- Migration written, applied to a copy of production, and reversible by a documented roll-forward.
- Feature degrades when its keys are missing (checked through `features()` and shown in Setup).
- Authorisation test: another user cannot read or change this data.
- Tests added for each rule in the spec; CI green.
- README and SETUP.md updated; new environment variables listed in `.env.example`.
- Privacy: new data listed in the privacy page and covered by account deletion.

9 Decisions and open questions

Decisions this spec makes

#	Decision	Why
D1	Fingerprints exclude line numbers and counts	Diff, triage and SARIF all need identity that survives edits
D2	Triage applied at read time, not inside the analysis	Keeps the shared public cache user-neutral
D3	Triage adjusts the score retroactively, with a raw toggle	Honest trends; no cliff when a team triages a backlog
D4	Pre-existing issues never fail a PR check	Adoptable on old code
D5	Background work runs from a Postgres queue drained by cron	No new infrastructure; fits Vercel
D6	GitHub App for repo access, OAuth app kept for sign-in	Installation tokens do not depend on a person staying employed or signed in
D7	Contents write is requested only for AI fix PRs	Smallest permission first
D8	PDF via print stylesheet, not a PDF library	Zero server cost, good enough layout
D9	Churn from GraphQL for the top 30 files, not full history	One call instead of hundreds; risk-relevant files only
D10	Scores stay comparable: churn changes file risk, not the health score	Trends remain valid across the release
D11	Tests and CI moved to Phase 0	Everything else is built on rules that need regression protection

Questions for you

- **Order:** start with M0 to M3 (everything stays inside the app), or jump to M4 and M5 (PR comments) first? Recommendation: M0 to M3, because M5 reuses the diff.
- **Free tier:** are 3 private repositories and no automatic analysis the right free limits?
- **Alerts:** Slack first, email first, or both in v1?
- **AI code access:** is 'snippet sent to the AI on explicit click' acceptable for chat and fix PRs, or must the AI never see code?
- **Naming:** keep the OAuth app and add a GitHub App, or migrate sign-in to the App's user authorisation to have one app?
- **Languages:** after JS, TS and Python, which comes first: Go, Java or Rust?

Part 3 - Build plan

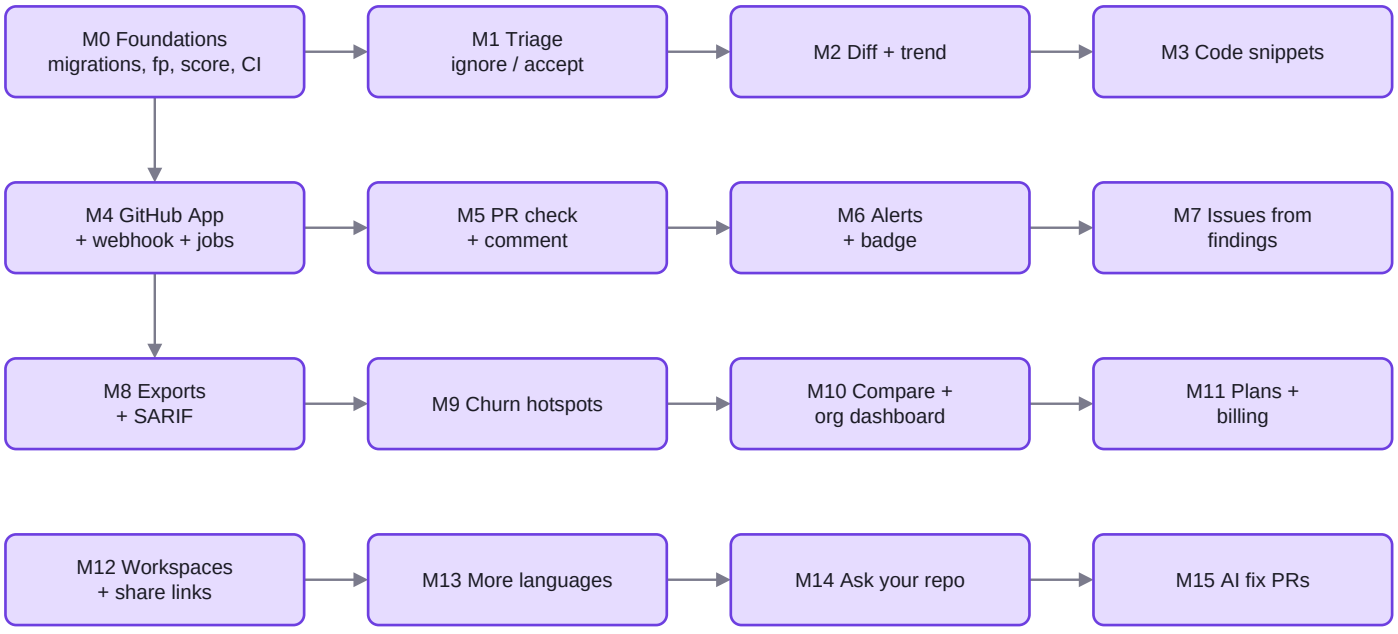
Plan and releases

The order, scope and checklist for building everything in this document. It follows the milestones in section 8 of the spec and adds tasks, key tests, effort and releases. Effort is a rough size for one developer: **S** about 3 to 5 days, **M** 1 to 2 weeks, **L** 2 to 4 weeks. Treat it as relative sizing, not a promise.

Releases

Release	Milestones	What users get	Why this grouping
1 Actionable	M0 to M3	Triage, diff, score trend, code snippets	Everything stays inside the app; no new infrastructure; M0 de-risks the rest.
2 Continuous	M4 to M7	Auto analysis, PR checks and comments, alerts, badge, issues	Needs the GitHub App and job queue; M5 reuses the diff from release 1.
3 Insight	M8 to M10	Exports and SARIF, churn hotspots, compare, org dashboard	Independent of each other; can be reordered or built in parallel.
4 Product	M11 to M15	Plans, teams, share links, more languages, chat, fix PRs	Commercial and team features, then AI features that depend on earlier ones.

Dependency map



Order is the recommended sequence. M2 must land before M5 (PR comments reuse the diff); M4 before everything automatic.

Critical path: M0, M1, M2, M4, M5, M11. Everything else can slide without delaying the commercial milestone.

Risks to watch

Risk	Where it bites	Mitigation built into the plan
GitHub rate limits	Automatic analysis on many repos	Installation tokens (5,000 per hour each); 2 calls per analysis; coalescing; backoff 1, 5, 15 minutes.
Serverless time limits	Analysis longer than the function limit	Analysis budget of 90 s inside a 300 s function; large repos skipped with a clear reason.
Unstable issue identity	Diff, triage, alerts all misfire	Fingerprints and tests in M0 before any feature relies on them.
Webhook duplicates and forgeries	Double comments, abuse	HMAC check, delivery-id dedupe, idempotent jobs, one comment through a marker.
Noisy findings	Users stop trusting the tool	Triage, false-positive tallies, only new issues fail a check, AI false-positive rating.
AI cost and privacy	Chat and fix PRs	Plan quotas, facts-only prompts, snippets only on explicit click and masked.
Data migrations	Workspaces move ownership	Additive migrations, personal workspace copy, numbers must not change (tested).

Definition of done for every milestone

- Migration applied to a copy of production; roll-forward documented.
- Feature degrades when its keys are missing and shows that in Setup.
- Authorisation test: another user cannot read or change the data.
- Tests for every rule in the spec; CI green.
- README, SETUP.md and .env.example updated; privacy page lists new data; account deletion covers it.
- Short note in the changelog saying what users can now do.

Milestone checklists

M0 Foundations

Phase 0 - Effort L - Depends on None. Make the codebase safe to change: migrations, stable issue identity, a pure score function, tests and CI. No visible change for users.

Tasks	[ ] Add src/server/migrate.ts and the prism_migrations ledger; write migration 0001 as the current schema; replace create-on-first-use in db.ts. [ ] Add rule, subject and fingerprint to Issue (analysisTypes.ts); supply subject from parse.ts, secrets.ts and deps.ts; assign occurrence index after sorting by path and line. [ ] Record overflow (issues dropped by the 300 cap and per-rule caps) in the result. [ ] Extract computeScore() into src/lib/score.ts; migration 0002 adds commit_full, branch_key, score_version, inputs and overflow to prism_analyses; save them in saveAnalysis(). [ ] Set up Vitest, tests/fixtures (tiny repos), a fetch mock, and a Postgres service in CI. [ ] Write unit tests for every rule in parse.ts, every pattern in secrets.ts, graph cycles, fingerprint stability, and score (golden and property tests). [ ] Add .github/workflows/ci.yml: lint, tsc --noEmit, vitest, next build; add the Playwright smoke test.
Key test	Fixture repo analysed twice gives identical fingerprints; moving code does not change them; score for an unchanged repo is identical before and after the refactor.
Ships when	CI green on main; fingerprints and inputs saved on every new analysis; existing databases adopted by the ledger.

M1 Triage

Phase 1 - Effort M - Depends on M0. Let a user ignore, mark false positive or accept the risk of a finding, and see the score change.

Tasks	[ ] Migration: prism_issue_states (user, repo, fingerprint, state, note, expires_at, issue_url). [ ] db.ts helpers: set, clear and list states for a repo; expire accepted states at read time. [ ] POST /api/triage with validation (state, note length, expiry). [ ] loadRepo.ts overlay: tag issues with their state and recompute the adjusted score from stored inputs. [ ] UI: row menu and dialog in SectionView, a Triaged tab, an adjusted/raw label on the score. [ ] Tests: overlay logic, expiry, per-user isolation, score with and without triage.
Key test	Ignoring a critical issue raises the score by 8 within caps; the state survives a commit that moves the line; another user is unaffected.
Ships when	A signed-in user can triage any finding and the dashboard reflects it immediately.

M2 Diff and trend

Phase 1 - Effort M - Depends on M0, M1. Show what changed between two analyses and how the score moved over time.

Tasks	[ ] src/lib/diff.ts: classify new, fixed, unchanged and changed by fingerprint; exclude triaged; flag approximate when overflow or old rows. [ ] Diff page /app/r/[owner]/[repo]/diff and the compare picker in History. [ ] Raise repoHistory limit to 50; trend chart as inline SVG with an accessible table; raw/adjusted toggle and range filter. [ ] Handle first analysis, different branches and different scoring versions with banners. [ ] Tests: each classification, self-diff is empty, moved code gives an empty diff, overflow flag.
Key test	Fixing one issue and adding one gives exactly one Fixed and one New; chart points equal the table.
Ships when	History shows a chart and a working two-run comparison.

M3 Code snippets

Phase 1 - Effort S - Depends on M0. Show the offending lines next to each finding.

Tasks	[ ] GET /api/snippet: contents API at commit_full with the viewer's or server token; normalise line endings. [ ] Reuse the scanner's masker so secret values never appear; cache an hour by (owner, repo, sha, path). [ ] UI: expandable snippet with the flagged line highlighted and an Open on GitHub link. [ ] Tests: masking on every secret rule, missing file, rate limit, large and binary files.
Key test	The highlighted line equals the finding's line; no secret value appears in any response.
Ships when	Every line-level finding can show its code.

M4 GitHub App, webhooks, jobs

Phase 2 - Effort L - Depends on M0. Run analyses automatically when code changes, reliably and in the background.

Tasks	[ ] Register the GitHub App (contents read, metadata read, pull requests write, checks write, issues write); store app id, private key and webhook secret. [ ] src/server/githubApp.ts: sign a JWT with jose, fetch and cache installation tokens. [ ] Migrations: prism_installations, prism_deliveries, prism_jobs, repo_settings.auto_analyze; /app/setup/github page and installation events. [ ] POST /api/github/webhook: verify HMAC on the raw body, dedupe by delivery id, enqueue with coalescing, answer 202. [ ] POST /api/jobs/run: CRON_SECRET, claim with for update skip locked, priority, retries 1, 5, 15 minutes, reaper for stuck jobs; cron in vercel.json. [ ] Setup page: last delivery, queue depth and last failure. [ ] Tests: signature (valid, invalid, missing), duplicate delivery, two runners, retry timing, stale requeue.
Key test	A forged signature creates nothing; killing a runner mid-job leads to exactly one saved analysis.
Ships when	A push to a connected repo produces a saved analysis without anyone clicking.

M5 PR check and comment

Phase 2 - Effort L - Depends on M2, M4. Report what a pull request does to health, on the pull request.

Tasks	[ ] src/lib/policy.ts: the seven ordered rules (secret, critical, minimum score, maximum drop, high, no baseline, success). [ ] Baseline resolution: analysis of the base sha, or enqueue it first with depends_on. [ ] githubChecks.ts: create check in progress, complete with title, summary and up to 50 annotations. [ ] prComment.ts: one comment updated through a hidden marker. [ ] Repo settings page with thresholds and a preview over stored PR analyses. [ ] Tests: each policy rule, annotations, comment upsert, fork and missing-baseline paths.
Key test	A PR adding one eval fails the check and shows one new critical; removing it passes; ten pushes keep one comment.
Ships when	Pull requests get a check and a single comment.

M6 Alerts and badge

Phase 2 - Effort M - Depends on M4. Tell people on Slack or email, and give repositories a README badge.

Tasks	[ ] Migrations: prism_alert_channels, prism_alert_log; encrypt targets with DATA_KEY. [ ] alerts.ts: events, dedupe key, one message per channel per repo per hour with a digest line; Slack Block Kit and Resend email; retry twice, disable after three failures. [ ] Settings, Alerts page with a test button; Slack URLs validated against hooks.slack.com/services. [ ] GET /api/badge/[owner]/[repo]: SVG from a template, colour by grade, token for private repos, cache headers, rate limit. [ ] Tests: dedupe, throttle, encryption round trip, badge colours and private-token rule.
Key test	A committed key posts to Slack within two minutes; replaying the webhook sends nothing more.
Ships when	Alerts work for new critical issues, secrets and score drops; badges render.

M7 Issues from findings

Phase 2 - Effort S - Depends on M4. Turn a finding into a tracked GitHub issue.

Tasks	[ ] githubIssues.ts and POST /api/issues with a preview dialog (labels, assignees). [ ] Store issue_url on the triage row; refuse duplicates for the same fingerprint. [ ] On a later analysis where the fingerprint is gone, comment 'Fixed in SHA'; close only if the setting is on. [ ] Tests: duplicate guard, masked secret body, issues disabled on the repo.
Key test	Clicking twice creates one issue; the permalink opens the analysed line.
Ships when	One click creates a well-formed GitHub issue.

M8 Exports and SARIF

Phase 3 - Effort M - Depends on M0. Take results out of PRISM in useful formats.

Tasks	[ ] GET /api/export with md, json, csv and sarif; secrets masked everywhere. [ ] Print-ready report page with a print stylesheet. [ ] src/lib/sarif.ts: 2.1.0 mapping with partialFingerprints from the fingerprint; optional upload to code scanning. [ ] Tests: SARIF validates against the official schema; large-result truncation note.
Key test	The Security tab shows the same issues as PRISM after an upload.
Ships when	All formats downloadable; SARIF accepted by GitHub.

M9 Churn hotspots

Phase 3 - Effort S - Depends on M0. Rank files that are both complex and frequently changed.

Tasks	[ ] analysis/churn.ts: one GraphQL query with 30 aliased history fields for the top files. [ ] File risk version 2: 35 complexity, 35 issues, 12 fan-in, 18 churn; health score unchanged. [ ] UI: churn column and map colouring; note when no token is available. [ ] Tests: ranking with and without churn; GraphQL failure handling.
Key test	A complex, often-changed file outranks a complex but untouched one.
Ships when	Hotspots reflect change frequency.

M10 Compare and org dashboard

Phase 3 - Effort M - Depends on M2. See all repositories at once and compare them.

Tasks	[ ] Query: latest analysis per repo with distinct on; aggregates in SQL; pagination at 50. [ ] /app/repos table with sparkline, critical count, staleness and failing runs; worst offenders list. [ ] /app/compare for up to 4 repos with normalised metrics. [ ] Tests: aggregates equal row sums; scoring-version banner.
Key test	Totals equal the sum of the rows; each row opens its repo.
Ships when	A single page answers which repositories need attention.

M11 Plans and billing

Phase 4 - Effort L - Depends on M5. Enforce limits from one function and take payment.

Tasks	[ ] Migrations: prism_subscriptions, prism_usage; src/server/plans.ts with can(). [ ] Stripe Checkout, Customer Portal and webhook (signature check, idempotent by event id, past_due grace of 7 days). [ ] Enforcement points: private repo count, automatic analysis repos, alerts, chat and fix quotas, share links, retention. [ ] Pricing page and Settings, Billing; downgrade behaviour that never deletes data. [ ] Tests: every limit, upgrade, downgrade, replayed Stripe events.
Key test	A Free user at the limit sees which limit and how to raise it; replaying an event changes nothing.
Ships when	Plans enforced and purchasable.

M12 Workspaces and share links

Phase 4 - Effort L - Depends on M11. Move ownership from person to team and let reports be shared safely.

Tasks	[ ] Migrations: prism_workspaces, prism_members, workspace_id on scoped tables; copy each user's data into a personal workspace. [ ] workspace.ts resolver used by every query; roles owner, admin, viewer; invitations bound to a GitHub login. [ ] prism_shares, /s/[token] public snapshot, revoke, expiry, noindex, view count; private-repo file-name hiding. [ ] Tests: authorisation matrix across roles and workspaces; migration changes no existing numbers.
Key test	A viewer cannot change settings; revoking a share fails the link within a minute.
Ships when	Teams share repos, settings and reports.

M13 More languages

Phase 4 - Effort L - Depends on M0. Add Go, Java and Rust and parse Python properly.

Tasks	[ ] Language interface and registry; web-tree-sitter with lazy-loaded WASM grammars. [ ] Starter rules per language (string-built SQL, command execution, ignored errors, unsafe and unwrap). [ ] Import resolution per language; skip generated code. [ ] Replace the line-based Python parser; review score drift on existing fixtures. [ ] Tests: positive and negative fixture for every rule.
Key test	Existing fixture scores move by at most 2 points after the Python change, and every move is reviewed.
Ships when	Go, Java and Rust repositories are analysed.

M14 Ask your repo

Phase 4 - Effort M - Depends on M2. Answer questions about an analysis with citations.

Tasks	[ ] Server tools: list_issues, get_issue, get_file_facts, get_diff, get_hotspots (read-only). [ ] POST /api/chat with streaming and the existing fact-only system prompt. [ ] Quota per plan; snippet only on explicit click, masked. [ ] UI in AI Review; conversation kept in the browser. [ ] Tests: every cited id exists; unknown file is reported as unknown.
Key test	Every answer naming a problem cites a valid issue id.
Ships when	Users can ask what to fix first and why.

M15 AI fix PRs

Phase 4 - Effort M - Depends on M7, M14. Propose a fix and open it as a reviewable pull request.

Tasks	[ ] Per-repo opt-in for sending snippets; request contents write permission only now. [ ] fixPr.ts: model call with the masked snippet; validate the patch applies, parses and no longer triggers the rule. [ ] Create branch and commit through the Git data API; open the PR with explanation and risk note; never merge. [ ] Monthly quota; no fix PR for secrets. [ ] Tests: rejected patches are never offered; PR passes PRISM's own check for that finding.
Key test	A proposal that does not apply is never shown.
Ships when	A finding can become a ready-to-review pull request.